

Figure-by-figure evaluation of the evidence

Every panel of the 5 main and 6 Extended Data figures, regenerated with every run scored as the public results site shows it, one panel at a time (43 panels): the question each panel answers, how it was generated, and what its evidence shows.

Prepared 7 October 2026

Scores as shown on goeckslab.github.io/galaxy-agent-benchmark on 7 October 2026

Run archive at commit [bf097f148](https://github.com/goeckslab/galaxy-agent-benchmark/commit/bf097f148) + uncommitted changes (archive/[reruns-token-figures-2026-10-07](https://github.com/goeckslab/galaxy-agent-benchmark/tree/main/archive/reruns-token-figures-2026-10-07)); figures in [result_evaluation/figures/](#)

How to read this document

Each panel has its own page with four parts:

- **Title.** the question the panel answers.
- **Panel.** the panel image, redrawn alone from the data its figure script recorded.
- **Rationale.** how the panel was generated: the data and their source (**Data**), what is plotted (**Variables**), the estimators, intervals and tests (**Analysis**), and how to read the encodings (**Reading the plot**).
- **Conclusion.** what the evidence shows in answer to the title, and where its inference stops.

The figures were regenerated for this document in `result_evaluation/figures/`, with the manuscript figures' own drawing and statistics code. Every run carries the score the public results site displays: BixBench-Verified-50 as graded on the site (two items, bix-53-q2 and bix-43-q2, regraded after the original evaluation; 27 runs), CompBioBench as on its official leaderboard, and IWC as shown per run, now including the host-read removal task. The 3,840 primary runs on 160 tasks are all scored. Run results were checked against the traces, submitted files and Galaxy histories. The repository's figures/ holds the same set; `result_evaluation/figures/README.md` lists every difference from the earlier set (`figures/archive/2026-10-07_before_site_scores/`).

The panels were not cropped from the figure images. `split_panels.py` redraws each composite figure from the panel data its script recorded and saves each panel alone; each redrawn composite matches its PNG pixel for pixel. Every number in the text comes from the figure's Source Data file (`result_evaluation/figures/*_source_data.csv`), its legend or the panel itself.

SHARED DEFINITIONS

Conditions	Custom code: the agent installs software and writes and runs its own code in an isolated workspace. Galaxy: the agent runs installed Galaxy tools and agent-written user-defined tools (UDTs) as jobs on usegalaxy.org through a Model Context Protocol (MCP) interface. Prompts, containers and time limits differ, so the conditions are compared as deployed.
Benchmarks and scores	BixBench-Verified-50: 50 questions graded as on the results site, the original evaluator's acceptance with documented regrades of bix-53-q2 (directional answer accepted) and bix-43-q2 (platform-specific rounding). CompBioBench: 100 questions scored against a reconstructed answer key that reproduces the official leaderboard. IWC: 10 workflow tasks scored by agreement of output files with curated workflow outputs (0–1), against the reference of the declared route where the method is open. Scores are benchmark-specific and never pooled.
Correct run	Accepted (BixBench-Verified-50, CompBioBench) or, for IWC, output agreement ≥ 0.99 .
Models and replicates	Four model configurations (GPT-5.5, GPT-5.6 Sol, GPT-5.6 Luna, DeepSeek V4 Pro) on the Codex agent harness. Each ran every task three times per condition: independent repeats, not successive attempts. A replicate set is the three runs of one task by one model in one condition; a task–model pair is the two replicate sets of one task and model.
Intervals	95% percentile cluster-bootstrap intervals (20,000 resamples unless stated); clusters are BixBench source capsules, otherwise tasks. Wilson intervals for small coded samples.
P values	Paired cluster sign-flip or randomization tests (200,000 draws; exact enumeration for IWC), Holm-adjusted within each family. A non-significant test does not establish equivalence; no equivalence margin was prespecified.

Contents

Figure 1 Study design and isolated execution pipeline.....	5
1a Which benchmarks, conditions and model configurations were compared, and how many runs and scores does the design yield?.....	6
1b What does every run go through, and which records does it leave for scoring and audit?.....	7
Figure 2 Agents show similar observed benchmark performance in Galaxy and custom code.....	8
2a Do the four models score similarly in Galaxy and in custom code on each benchmark?.....	9
2b How large is the Galaxy minus custom-code difference for each model and for the four models pooled?.....	10
2c For the same task and model, do the two conditions get the same number of correct runs out of three?.....	11
2d What caused the incorrect BixBench-Verified-50 runs where the two conditions disagree, and where they agree?.....	12
Figure 3 Galaxy provides a structured environment for agent analyses.....	13
3a Does running in Galaxy change the share of runs correct in any task domain?.....	14
3b Which Galaxy routes did agents use in each run, and how often were those runs correct?.....	15
3c How often did each kind of execution step fail, what kinds of error occurred, and were failed steps fixed later?.....	16
3d Were runs with execution errors more often correct in Galaxy than in custom code?.....	17
3e When agents requested installed tools, did Galaxy's record match the parameters they asked for?.....	18
3f Which infrastructure changes could prevent the most failed Galaxy requests?.....	19
Figure 4 Answer agreement and tool use vary across model configurations.....	20
4a Which kinds of Galaxy tools did each model run on each benchmark?.....	21
4b How often did three replicate runs give the same answer, and do models differ?.....	22
4c Are tasks whose replicate runs used the same Galaxy tools solved more often?.....	23
4d Did correct runs check their own analyses more often than incorrect runs?.....	24
4e When replicate runs fail, do they repeat the same rejected answer, and does that track task difficulty?.....	25
Figure 5 Galaxy records analyses as structured provenance and uses more input tokens on question-answering tasks.....	26
5a Does Galaxy use more tokens than custom code on each benchmark, and do incorrect runs use more input than correct ones?.....	27
5b Does Galaxy's extra input come from more actions or from more input per action, and what does Galaxy return to the agent?.....	28
5c Can changes to the Galaxy interface reduce Galaxy's token use relative to custom code?.....	29
5d What does the retained record hold for each analysis step in each condition?.....	30
5e How does the same analysis step appear in the custom-code trace and in the Galaxy job record?.....	31
Extended Data Fig. 2 Failure causes and sensitivity of the accuracy comparison.....	32
ED 2a What caused each incorrect BixBench-Verified-50 run, by how many runs of its replicate set failed?.....	33
ED 2b Does the Galaxy minus custom-code difference change with the IWC correctness threshold or the population analysed?.....	34
Extended Data Fig. 3 Task status, execution errors and final correctness.....	35
ED 3a Are failures spread across tasks, or concentrated in a few tasks that fail in both conditions?.....	36
ED 3b Which kinds of execution error occurred in each channel?.....	37
ED 3c Does the relation between execution errors and correctness in Fig. 3d hold within each benchmark?.....	38
Extended Data Fig. 4 Tool inventory, route similarity, answer matching, difficulty and UDT methods.....	39
ED 4a Which installed Galaxy tools were run most often, and by which models?.....	40
ED 4b Do models differ in tool-set similarity, and does the Fig. 4c correlation hold under other route definitions?.....	41
ED 4c Does the answer-matching rule change how often replicate runs agree?.....	42
ED 4d Does the difficulty pattern of Fig. 4e hold when the same model's runs are left out of difficulty?.....	43
ED 4e What did the completed UDT jobs compute?.....	44
Extended Data Fig. 5 Token use by model, outcome and action count.....	45
ED 5a Does higher input-token use go with higher scores across model configurations and conditions?.....	46
ED 5b Do incorrect runs use more input tokens than correct runs, model by model?.....	47

ED 5c How does a run's input-token use scale with its number of actions?.....	48
Extended Data Fig. 6 Independent checks of the audits and of benchmark integrity.....	49
ED 6a How many runs reached a source of benchmark answers during the run, and in which benchmark and condition?.....	50
ED 6b Does the Galaxy–custom code difference in runs correct change when runs that reached or searched for benchmark answers are removed?.....	51
ED 6c Does an independent rater assign the same primary failure cause as the original audit to incorrect BixBench-Verified-50 runs?.....	52
ED 6d Does an independent rater reproduce the rule-based failure classes of failed Galaxy requests?.....	53
Extended Data Fig. 7 Verification, recovery and selected cases.....	54
ED 7a How often did agents check their own analysis, and did checks differ between correct and incorrect runs or between conditions?.....	55
ED 7b How often was a failed step re-run without error later in the same run, and did those runs end correct more often?.....	56
ED 7c What happens when the Galaxy interface detects that a request would not set the parameter the agent asked for?.....	57
ED 7d In a task where read-end artefacts mimic an alternate allele, which runs ran a read-position diagnostic, and which were accepted?.....	58

Figure 1 | Study design and isolated execution pipeline

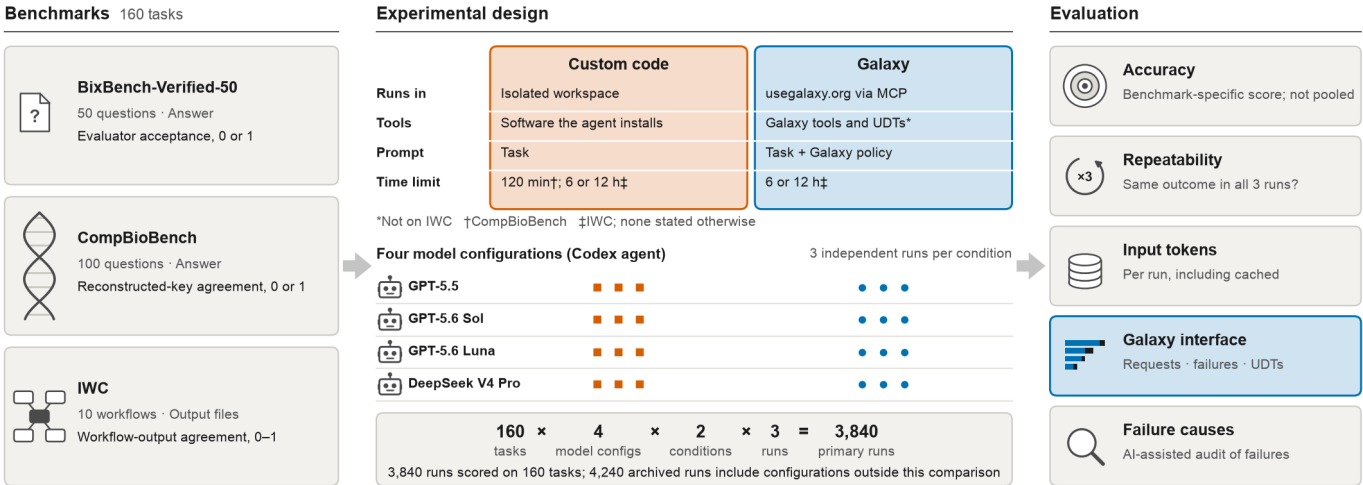
Figure 1 sets out what the study compares and how each run is produced and judged. It shows whether the custom code and Galaxy conditions are balanced in tasks, models and repeats, and which records each run leaves for scoring and audit. Every later figure depends on these choices.

PANELS IN THIS FIGURE

- **Fig. 1a** Which benchmarks, conditions and model configurations were compared, and how many runs and scores does the design yield?
- **Fig. 1b** What does every run go through, and which records does it leave for scoring and audit?

Fig. 1a | Which benchmarks, conditions and model configurations were compared, and how many runs and scores does the design yield?

a Study design



Panel a of result_evaluation/figures/fig1_a.pdf · image: result_evaluation/panels/fig1a.png

RATIONALE

- **Data.** The schematic summarises the design. The script reads the task counts from the repository (50 BixBench-Verified-50 questions from 33 analysis capsules, 100 CompBioBench questions across eight domains, 10 IWC workflow tasks) and the 4,240 archived runs from the per-run design metadata. The unit is one run: one task, model configuration, condition and replicate.
- **Variables.** Left: the three benchmarks and their scoring (evaluator acceptance, 0 or 1; reconstructed-key agreement, 0 or 1; workflow-output agreement, 0–1). Centre: where each condition ran the analysis, its tools, prompt and time limit, then four model configurations on the Codex agent with three independent runs per condition. Right: the five evaluation outputs.
- **Analysis.** None (schematic); counts only. 160 tasks × 4 model configurations × 2 conditions × 3 runs gives 3,840 primary runs, all of them scored. The IWC host-read removal task is scored from each run's run_record.json, the value the results site shows.
- **Reading the plot.** Read left to right; orange squares are custom-code runs and blue circles Galaxy runs. Footnote marks show that UDTs were not offered on IWC, the 120-min limit applies to CompBioBench custom-code runs and the 6 or 12 h limit to IWC.

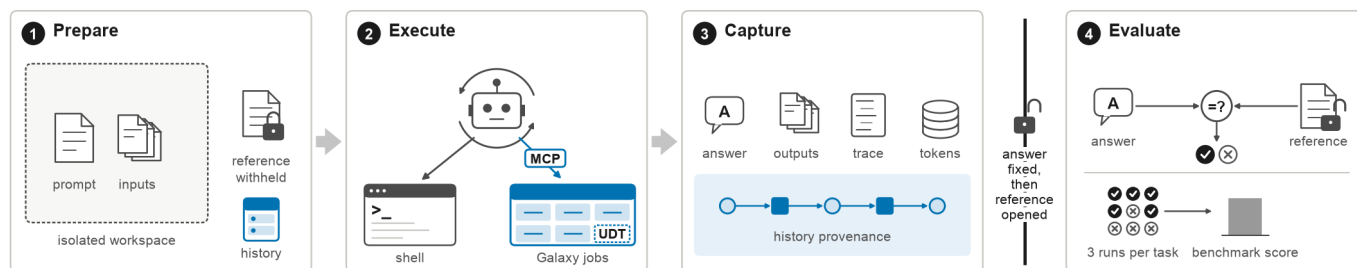
CONCLUSION

The design crosses 160 tasks with four model configurations, two conditions and three independent runs. This gives 3,840 primary runs, all of them scored. It is balanced in tasks, models and repeats. However, the conditions also differ in prompt (Galaxy adds an execution policy), container or environment, and stated time limit, so they are compared as deployed, not as a single-factor contrast. Replicates measure repeatability, not successive attempts. Scores are benchmark-specific and are not pooled. For CompBioBench, some replicate sets combine runs from different campaigns. One IWC score is self-referential: on the host-read removal task, the minimap2 reference output is byte-identical to the submission of GPT-5.5 custom-code replicate 3.

Fig. 1b | What does every run go through, and which records does it leave for scoring and audit?

b Isolated execution pipeline applied to each of the 3,840 primary runs

Galaxy condition only



Panel b of result_evaluation/figures/fig1_a.pdf · image: result_evaluation/panels/fig1b.png

RATIONALE

- **Data.** The schematic shows the pipeline applied to each of the 3,840 primary runs. It summarises the harness and the scoring procedure, not a measured quantity.
- **Variables.** Prepare puts the prompt and inputs in an isolated workspace, withholds the reference and, for Galaxy runs, creates a history; Execute runs shell commands, or Galaxy jobs and UDTs through MCP. Capture keeps the answer, output files, trace and token counts, plus history provenance for Galaxy runs; Evaluate compares the answer with the reference, and each task's three runs feed the benchmark score.
- **Analysis.** None (schematic).
- **Reading the plot.** Read left to right; blue marks elements of the Galaxy condition only. The lock between Capture and Evaluate is the gate: the reference is opened only after the answer is fixed. In Evaluate, the grid of ticks and crosses shows each run scored on its own before the runs are pooled into a benchmark score.

CONCLUSION

Every run follows the same four steps. The reference is opened only after the answer is fixed, so scoring cannot feed back into the answer. Galaxy runs also keep history provenance (datasets, tools, versions, parameters and job states). Custom-code runs keep the answer, trace and token counts, but the benchmark did not retain the files their analysis steps wrote (Fig. 5d), and the CompBioBench host environment was not exported. Withholding the reference from the workspace did not block web access to public benchmark sources. Some runs reached benchmark answers during the run (Extended Data Fig. 6a).

Figure 2 | Agents show similar observed benchmark performance in Galaxy and custom code

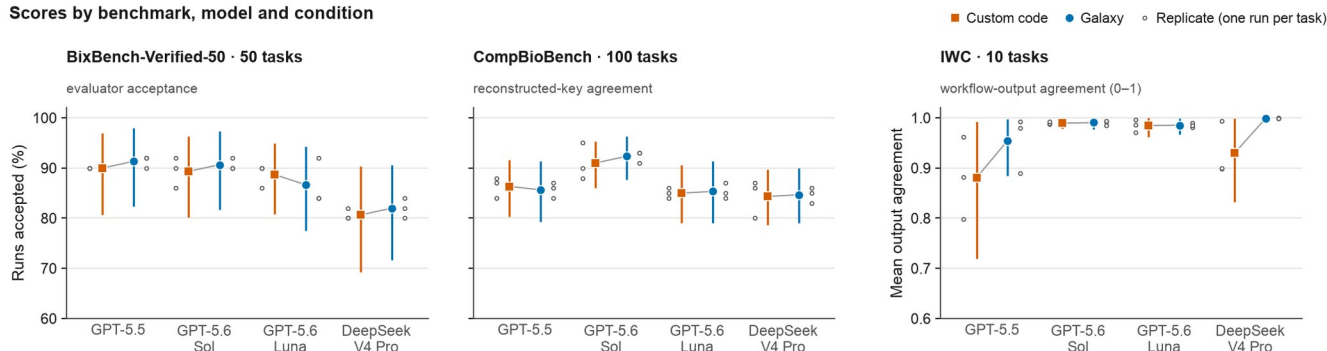
This figure asks whether running an analysis through Galaxy, rather than in custom code, changes how often the same four models reach the benchmark answer. It tests this on three benchmarks with different scoring contracts, as paired differences, and then asks why runs fail where the two conditions disagree. The answer frames the rest of the study: Galaxy's provenance and tool-use properties matter only if using Galaxy does not by itself cost accuracy.

PANELS IN THIS FIGURE

- **Fig. 2a** Do the four models score similarly in Galaxy and in custom code on each benchmark?
- **Fig. 2b** How large is the Galaxy minus custom-code difference for each model and for the four models pooled?
- **Fig. 2c** For the same task and model, do the two conditions get the same number of correct runs out of three?
- **Fig. 2d** What caused the incorrect BixBench-Verified-50 runs where the two conditions disagree, and where they agree?

Fig. 2a | Do the four models score similarly in Galaxy and in custom code on each benchmark?

a Scores by benchmark, model and condition



RATIONALE

- **Data.** All 3,840 scored primary runs, scored as shown on the public results site: BixBench-Verified-50 (50 tasks; the original evaluator's acceptance, with bix-53-q2 and bix-43-q2 regraded, changing 27 runs), CompBioBench (100 tasks) and IWC (10 tasks, including host-read removal), each run by four models in both conditions with three replicates. Each point summarizes one model in one condition: 150, 300 or 30 runs.
- **Variables.** The y axis is each benchmark's own score: the share of runs accepted (BixBench-Verified-50), the share agreeing with the reconstructed answer key (CompBioBench), and mean agreement with curated workflow outputs on a 0–1 scale (IWC). Orange squares are custom code and blue circles Galaxy; small open dots are replicate means (one run of every task).
- **Analysis.** The estimator is the mean score over the model's runs in that condition. Intervals are 95% percentile cluster-bootstrap intervals (20,000 resamples; clusters are BixBench source capsules, otherwise tasks). Descriptive only; the paired tests are in panel b.
- **Reading the plot.** A grey line joins the two conditions of one model, so its slope shows the direction of the difference. Vertical bars are 95% intervals. IWC has its own 0–1 axis, so heights are not comparable across facets.

CONCLUSION

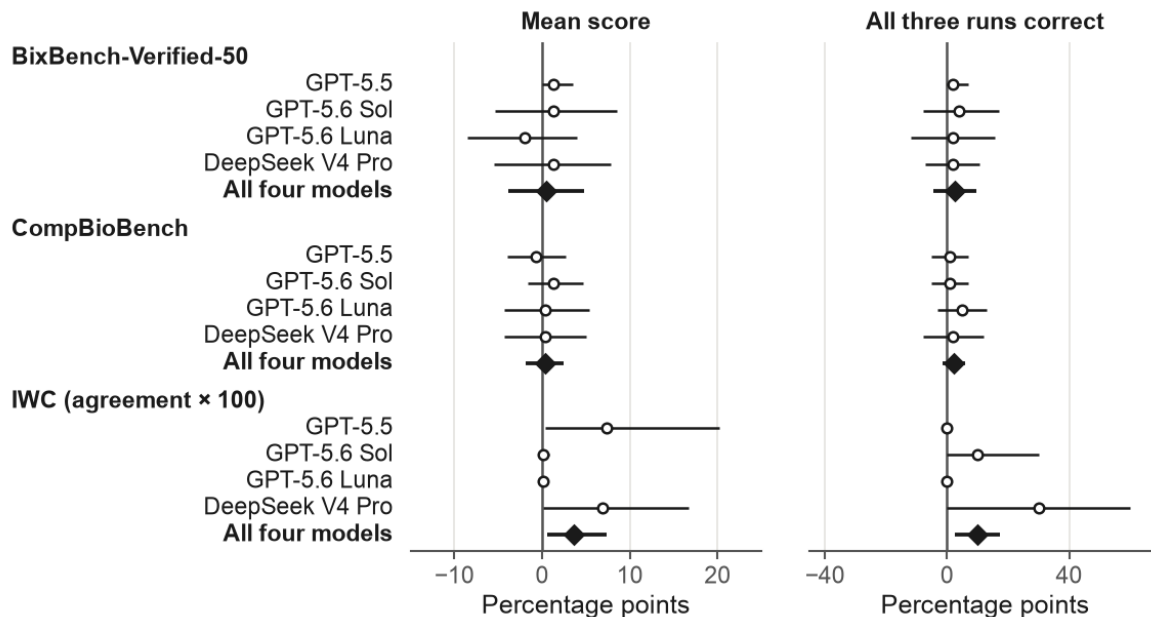
Yes, on two benchmarks. On BixBench-Verified-50 and CompBioBench, each model's two conditions differ by at most 2.0 points, and the intervals overlap widely (for example, GPT-5.5 on BixBench-Verified-50: 90.0% in custom code, 91.3% in Galaxy). On IWC, GPT-5.6 Sol and GPT-5.6 Luna sit near ceiling in both conditions (0.98 to 0.99). GPT-5.5 (0.88 vs 0.95) and DeepSeek V4 Pro (0.93 vs 1.00) score higher in Galaxy, but their custom-code intervals are wide (0.72 to 0.99; 0.83 to 1.00) and their replicate dots spread widely. This panel does not test the differences; panel b does.

Fig. 2b | How large is the Galaxy minus custom-code difference for each model and for the four models pooled?

b Paired differences: Galaxy minus custom code

← custom code higher · Galaxy higher → (95% intervals)

Without the 94 runs that reached benchmark answers: BixBench-Verified-50 +0.6, CompBioBench +0.2 points



Panel b of result_evaluation/figures/fig2.pdf · image: result_evaluation/panels/fig2b.png

RATIONALE

- **Data.** The same 3,840 runs, paired by task and model; the reliability estimand uses the 1,280 replicate sets (one task × model × condition, three runs). The grey note drops 94 runs that a scan of the traces found had reached benchmark answers.
- **Variables.** The x axis is Galaxy minus custom code in percentage points. The left column is the mean score (share accepted, or IWC agreement × 100). The right column is the share of replicate sets with all three runs correct (IWC at agreement ≥ 0.99). Rows are the four models and their pooled estimate, grouped by benchmark.
- **Analysis.** Estimates are differences of pooled means with 95% percentile cluster-bootstrap intervals (20,000 resamples). *P* values come from paired cluster sign-flip tests on each cluster's summed task-level differences (200,000 draws; exact for IWC). Holm adjustment is within three families: 12 per-model mean-score tests, 6 pooled tests (runs correct and all three correct, per benchmark) and 12 per-model all-three-correct tests. The pooled IWC agreement difference is unadjusted; no equivalence test was run.
- **Reading the plot.** Open circles are single models, filled diamonds the four models pooled, and horizontal lines 95% intervals. Points right of the zero line mean Galaxy scored higher.

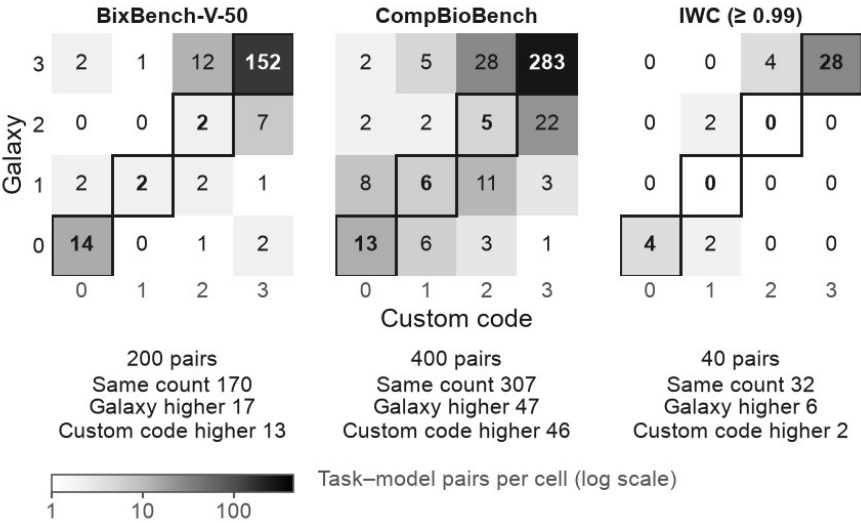
CONCLUSION

The differences are small, and no difference was significant after Holm adjustment (smallest adjusted *P* = 0.19, GPT-5.5 on IWC: +7.4 points, 0.4 to 20.1). Pooled mean-score differences were +0.5 points (−3.9 to 4.8; *P* = 0.88) on BixBench-Verified-50 and +0.3 (−1.8 to 2.5; *P* = 0.82) on CompBioBench; without the 94 exposed runs they were +0.6 and +0.2. The pooled IWC agreement difference, +3.6 points (0.6 to 7.4), has an exact *P* of 0.055; with ten tasks its interval is too narrow. The pooled intervals exclude Galaxy deficits larger than 3.9 and 1.8 points, but this does not establish equivalence.

Fig. 2c | For the same task and model, do the two conditions get the same number of correct runs out of three?

C Correct runs of three, same task and model

Outlined diagonal, same count; above it, Galaxy had more correct runs; below it, custom code had more



Panel c of result_evaluation/figures/fig2.pdf · image: result_evaluation/panels/fig2c.png

RATIONALE

- **Data.** All task-model pairs: 200 on BixBench-Verified-50 (50 tasks × 4 models), 400 on CompBioBench and 40 on IWC (10 tasks, including host-read removal). Each pair holds one replicate set per condition, scored as in panel a.
- **Variables.** Rows give the Galaxy set's correct runs (0–3) and columns the custom-code set's; each cell counts pairs. A run is correct when it was accepted or, for IWC, at agreement ≥ 0.99 . The text under each matrix gives the total pairs and the pairs with the same count, more correct runs in Galaxy, or more in custom code.
- **Analysis.** Descriptive only: counts of pairs, with no test. The paired tests on these sets are in panel b.
- **Reading the plot.** Grey shading encodes pairs per cell on a log scale; white cells are empty. The outlined diagonal (bold counts) marks equal counts; cells above it are pairs in which Galaxy had more correct runs, and cells below it pairs in which custom code had more. The top-right cell holds pairs correct in all six runs.

CONCLUSION

Usually, yes. Most pairs have the same count in both conditions: 170 of 200 on BixBench-Verified-50, 307 of 400 on CompBioBench and 32 of 40 on IWC, most of them correct in all six runs (152, 283 and 28). Discordant pairs split almost evenly on BixBench-Verified-50 (17 with Galaxy higher, 13 with custom code higher) and CompBioBench (47 vs 46), and lean toward Galaxy on IWC (6 vs 2). Some pairs failed every run in both conditions (14, 13 and 4). Equal counts mean equal replicate success, not identical answers or methods.

Fig. 2d | What caused the incorrect BixBench-Verified-50 runs where the two conditions disagree, and where they agree?



RATIONALE

- **Data.** The 151 BixBench-Verified-50 runs graded incorrect on the results site, each with a primary and optional secondary cause from an AI-assisted run-level audit of the traces; 4 bix-43-q2 runs made incorrect only by the site's regrade take the task-level audit's cause (benchmark specification or scoring). An independent AI second rater, blind to the audit, re-coded 45 runs sampled before the regrades (7 now graded correct); the box shows all 24 runs of task bix-45-q1.
- **Variables.** Pair types come from panel c: incorrect custom-code runs where Galaxy had more correct runs (17 pairs; 26 runs), incorrect Galaxy runs where custom code had more (13 pairs; 22 runs), and both conditions' incorrect runs in the 18 same-count pairs with any failure (48 runs each). Bars give each primary cause's share of the row; segment labels are run counts. The seven incorrect runs of the better condition in discordant pairs are left out of the bars.
- **Analysis.** Descriptive counts and shares; no test. The subtitle gives the second rater's raw agreement on the primary cause group and Cohen's κ .
- **Reading the plot.** Colour is the primary cause. Hatching marks runs whose primary and secondary causes were answer validation and benchmark specification. Text at right gives runs and tasks per row; in the box, each dot is one run (filled, correct; open, incorrect).

CONCLUSION

Shared failures dominate and are mostly attributed to the benchmark: in same-count pairs, 41 of 48 custom-code and 39 of 48 Galaxy failures were coded as benchmark specification or scoring. Where Galaxy did better, the custom-code failures were mostly missing answer validation (12 of 26) or specification (13). Where custom code did better, 3 of 22 Galaxy failures were coded as not able to use Galaxy; the traced case alone supplies 3 of these 13 pairs and 9 of their failures, coded as specification: a PhyKIT version difference encoded in the reference. The second rater agreed on 35 of 45 runs ($\kappa = 0.56$): these are moderately reliable attributions, not verified mechanisms.

Figure 3 | Galaxy provides a structured environment for agent analyses

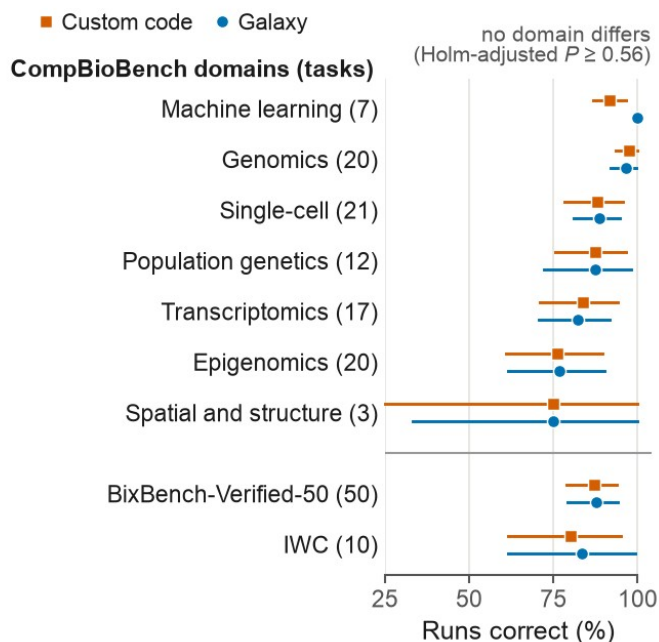
This figure asks what running analyses through Galaxy changed in how agents worked, beyond final correctness: whether correctness held in every task domain, how agents used Galaxy's jobs, how often steps failed and were fixed, and what Galaxy's job and parameter records reveal about failures. It matters because the study evaluates Galaxy as an execution environment. Its records should therefore show where agents used it, where it caught or caused problems, and which infrastructure changes might help.

PANELS IN THIS FIGURE

- **Fig. 3a** Does running in Galaxy change the share of runs correct in any task domain?
- **Fig. 3b** Which Galaxy routes did agents use in each run, and how often were those runs correct?
- **Fig. 3c** How often did each kind of execution step fail, what kinds of error occurred, and were failed steps fixed later?
- **Fig. 3d** Were runs with execution errors more often correct in Galaxy than in custom code?
- **Fig. 3e** When agents requested installed tools, did Galaxy's record match the parameters they asked for?
- **Fig. 3f** Which infrastructure changes could prevent the most failed Galaxy requests?

Fig. 3a | Does running in Galaxy change the share of runs correct in any task domain?

a Correct runs by domain



Panel a of result_evaluation/figures/fig3.pdf · image: result_evaluation/panels/fig3a.png

RATIONALE

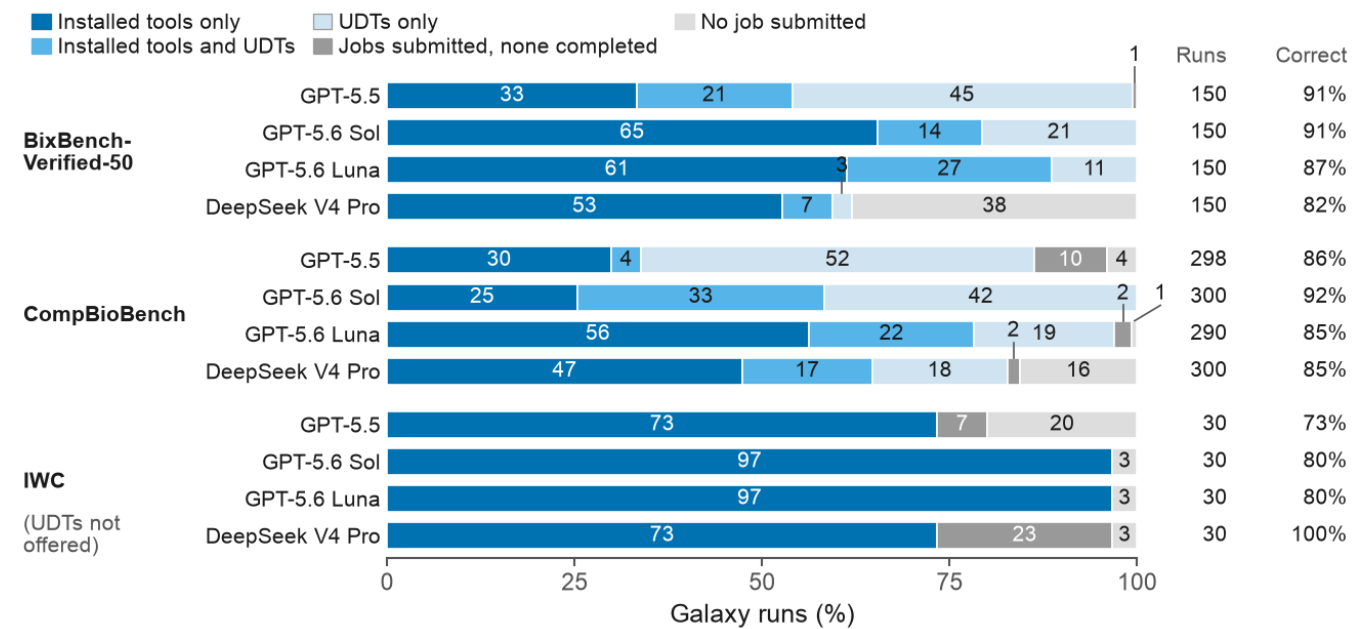
- **Data.** All 3,840 scored primary runs (160 tasks × four model configurations × three replicates × two conditions). BixBench-Verified-50 grades are those displayed on the results site (the original evaluator's, with documented regrades of bix-53-q2 and bix-43-q2), and IWC includes host-read removal. CompBioBench tasks are grouped by the benchmark's own domain labels (spatial and structure merged); BixBench-Verified-50 (50 tasks) and IWC (10 tasks) are shown whole. The unit is the run.
- **Variables.** x is the share of runs correct. A run counts as correct when the BixBench-Verified-50 or CompBioBench evaluator accepts it, or when IWC output agreement is at least 0.99; completing without error is not enough. Rows are domains, with CompBioBench domains sorted by the Galaxy estimate and task counts in parentheses.
- **Analysis.** The share correct per domain and condition, with 95% percentile cluster-bootstrap intervals (20,000 resamples; clusters are BixBench source capsules, otherwise tasks). Each domain's Galaxy minus custom-code difference is tested by a paired cluster randomization test (200,000 draws, or exact enumeration with 16 or fewer clusters), Holm-adjusted across the nine domains.
- **Reading the plot.** Orange squares, custom code; blue circles, Galaxy; horizontal lines, 95% intervals. The grey rule separates the CompBioBench domains from the other two benchmarks.

CONCLUSION

No domain showed a detectable difference between conditions (Holm-adjusted $P \geq 0.56$ for all nine). Point differences ranged from −1.5 points (Transcriptomics) to +8.3 points (Machine learning: Galaxy 100%, custom code 92%, 95% interval 87–96%; unadjusted $P = 0.06$). For BixBench-Verified-50, 87.7% (79.2–94.1%) of Galaxy runs and 87.2% (79.0–93.6%) of custom-code runs were correct. Intervals are wide for small domains (Spatial and structure, 3 tasks; IWC, 10 tasks). There, the panel cannot rule out moderate differences.

Fig. 3b | Which Galaxy routes did agents use in each run, and how often were those runs correct?

b How each Galaxy run used Galaxy, and how it ended



Panel b of result_evaluation/figures/fig3.pdf · image: result_evaluation/panels/fig3b.png

RATIONALE

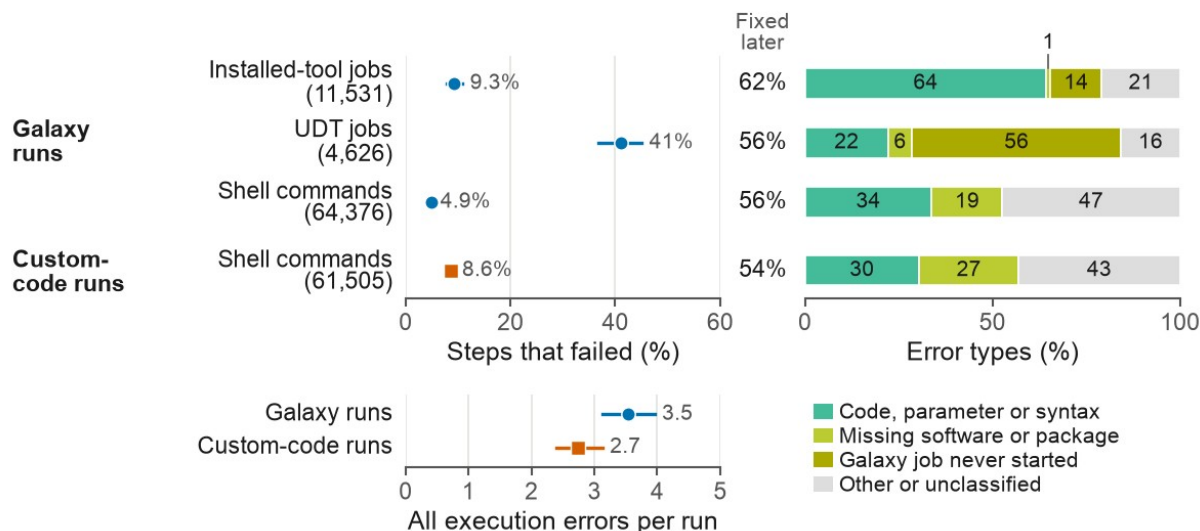
- **Data.** 1,908 traced Galaxy-condition runs of the four model configurations (600 BixBench-Verified-50, 1,188 CompBioBench, 120 IWC); twelve CompBioBench runs have no trace. Job submissions and job states come from the agent-interface call records extracted from the traces. Correctness comes from the grades of Fig. 3a (BixBench-Verified-50 as displayed on the results site; IWC including host-read removal, so every traced IWC run is scored). The unit is the run.
- **Variables.** Each run gets one route from the jobs it submitted through the interface's two job-running calls: completed installed-tool jobs only, both installed-tool and UDT jobs, UDT jobs only, jobs submitted but none completed, or no job. Bars give the share of each configuration's runs on each route. The right-hand columns give traced runs and the share of them correct.
- **Analysis.** Descriptive only. Correctness by route (pooled over models, in Source Data) is descriptive, because the agent chose the route.
- **Reading the plot.** One 100% bar per model configuration, grouped by benchmark; numbers are percentages of runs, with small segments labelled above the bar. Dark blue, installed tools only; mid blue, both; pale blue, UDTs only; dark grey, all jobs failed; light grey, no job (UDTs were not offered for IWC).

CONCLUSION

Routes varied more between model configurations than correctness did. On BixBench-Verified-50, GPT-5.6 Sol used installed tools only in 65% of runs, GPT-5.5 used UDTs only in 45%, and DeepSeek V4 Pro submitted no job in 38%. Yet the share correct ranged only from 82% to 91%. On CompBioBench, GPT-5.5 used UDTs only in 52% of runs. On IWC, 73–97% of runs used installed tools only. Pooled over models, BixBench-Verified-50 runs with no job were correct in 70% (57 runs), against 88–93% on routes with a completed job; this association is descriptive. The panel does not show where the analysis ran when no interface job completed.

Fig. 3c | How often did each kind of execution step fail, what kinds of error occurred, and were failed steps fixed later?

C How often execution steps failed, and were fixed later



Panel c of result_evaluation/figures/fig3.pdf · image: result_evaluation/panels/fig3c.png

RATIONALE

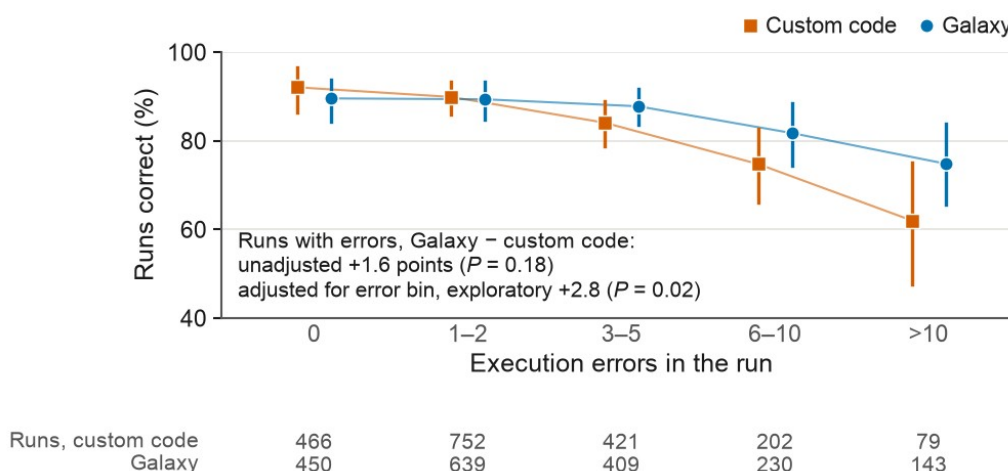
- **Data.** 3,791 scored runs with execution records (1,871 Galaxy, 1,920 custom code); shell commands come from the agent traces and the left-hand job shares from the agent-interface call records. Error types and errors per run come from the archive's per-error table, which takes Galaxy jobs in the error state from the archived job records, so its job-error counts differ from the left-hand failures. Errors of the 24 IWC host-read removal runs were added, extracted with the archive's own rules.
- **Variables.** Left: the share of steps that failed in each channel, with step counts in parentheses (a silent exit code 1 is not a failed shell command). "Fixed later" is the share of failed steps re-run without error later in the same run: a job of the same tool, a later UDT job, or the same named analysis program. Right, error types in four groups; below, all execution errors per run.
- **Analysis.** Shares and errors per run are ratios of sums with 95% percentile cluster-bootstrap intervals (20,000 resamples; BixBench source capsules, otherwise tasks, resampled within each benchmark). Descriptive only; no test.
- **Reading the plot.** Circles, Galaxy runs; squares, custom-code runs; lines, 95% intervals. Each stacked bar gives the percentage of that channel's errors in each type group.

CONCLUSION

UDT jobs failed most often: 41% (95% interval 36.8–45.0%), against 9.3% (7.9–10.8%) for installed-tool jobs. More than half of UDT errors (56%) were jobs that never started. Shell commands failed less often in Galaxy runs (4.9%, 4.6–5.1%) than in custom-code runs (8.6%, 7.9–9.3%). Even so, Galaxy runs had more errors per run (3.5, 3.1–4.0, against 2.7, 2.4–3.1); the intervals barely separate (Galaxy lower bound 3.14, custom-code upper bound 3.13), and no test was run. In every channel, 54–62% of failed steps were later re-run without error. A successful re-run does not show that the error was understood or that the result was right.

Fig. 3d | Were runs with execution errors more often correct in Galaxy than in custom code?

d Final correctness among runs with execution errors



Panel d of result_evaluation/figures/fig3.pdf · image: result_evaluation/panels/fig3d.png

RATIONALE

- **Data.** The same 3,791 scored runs with execution records as in Fig. 3c (1,920 custom code, 1,871 Galaxy), with the grades of Fig. 3a. Of these, 2,875 runs had at least one execution error. The unit is the run.
- **Variables.** x is the number of execution errors in the run (failed shell commands plus Galaxy jobs in the error state), in five bins: 0, 1–2, 3–5, 6–10 and more than 10. y is the share of runs correct. Run counts per bin and condition are printed below the axis.
- **Analysis.** Per-bin shares have 95% percentile cluster-bootstrap intervals (20,000 resamples). The primary estimate is the unadjusted Galaxy minus custom-code difference among runs with errors; an exploratory estimate averages it over the four error bins, weighted by pooled bin size, with bins chosen after inspection. Both *P* values come from paired cluster randomization tests that swap condition labels within clusters (200,000 draws).
- **Reading the plot.** Orange squares and line, custom code; blue circles and line, Galaxy; vertical lines, 95% intervals. The lines join the bin estimates only to guide the eye.

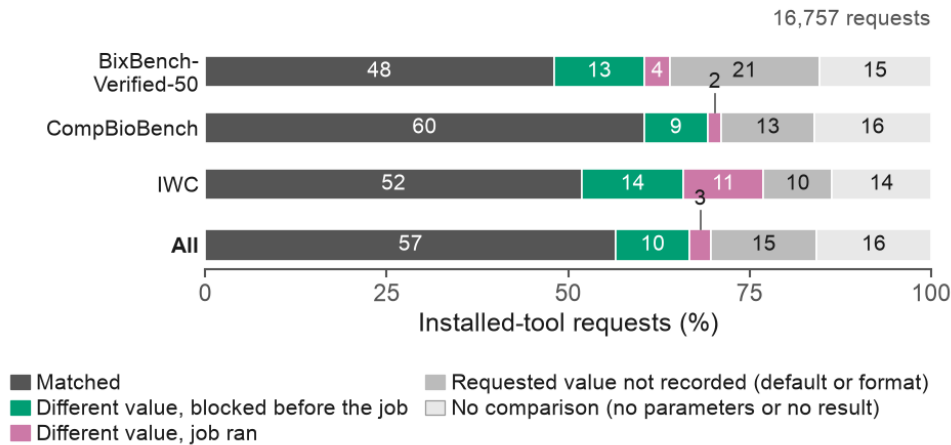
CONCLUSION

The primary comparison detected no difference. Among runs with errors, Galaxy runs were correct 1.6 points more often than custom-code runs ($P = 0.18$). The exploratory error-bin-adjusted difference was +2.8 points ($P = 0.02$). In both conditions, correctness fell as errors increased. The fall was larger in custom code (92.1% with no errors to 62.0% with more than 10) than in Galaxy (89.6% to 74.8%). Error counts are themselves outcomes of the run. The panel therefore shows an association, not recovery from individual failures.

Fig. 3e | When agents requested installed tools, did Galaxy's record match the parameters they asked for?

e Parameter checks on installed-tool requests

The interface compares the requested parameters with those Galaxy validated or recorded



After a request was blocked for a different value, the run later completed a job of the same tool in 71% of 1,700 cases (with matching parameters in 52%). Matching parameters do not show that the settings were scientifically appropriate.

Panel e of result_evaluation/figures/fig3.pdf · image: result_evaluation/panels/fig3e.png

RATIONALE

- **Data.** 16,757 installed-tool requests from traced Galaxy runs of the four model configurations (4,414 BixBench-Verified-50, 11,175 CompBioBench, 1,168 IWC), taken from the agent-interface call records. For each request, the interface compares the requested parameters with those Galaxy validated before the job or recorded after it. The unit is the request.
- **Variables.** Each request has one of five outcomes: matched; different value, blocked before the job; different value, job ran (Galaxy recorded a different value); requested value not recorded (no counterpart in Galaxy's record, as with defaults or reformatting); or no comparison (no parameters or no result). A follow-up measure records whether the run later completed a job of the same tool after a blocked request.
- **Analysis.** Descriptive only: counts and shares, with no intervals or tests.
- **Reading the plot.** One 100% bar per benchmark and one for all requests. Numbers are percentages, and small segments are labelled above the bar. The note below the bars gives the follow-up after blocked requests.

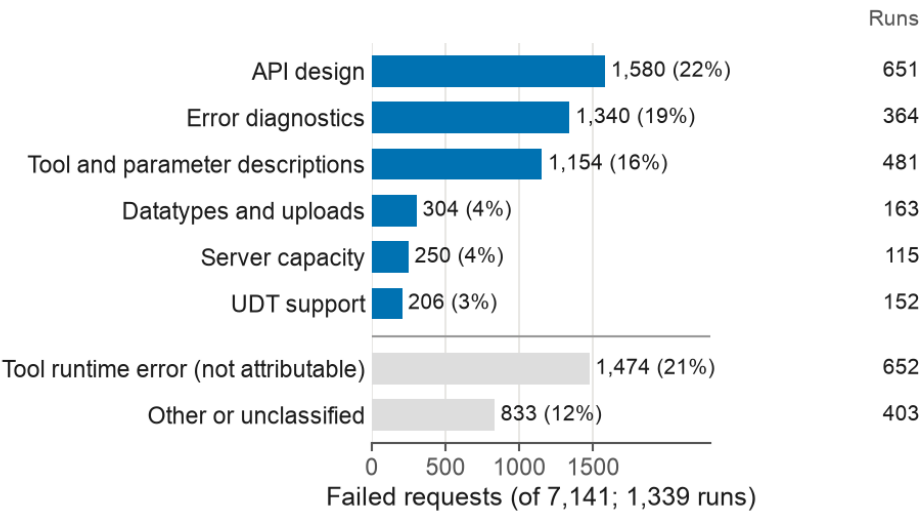
CONCLUSION

Galaxy's record matched the requested parameters for 57% of requests. A different value was caught before the job in 10% (1,700 requests) and recorded after the job ran in 3%. For 15%, Galaxy recorded no counterpart to the requested value, and 16% could not be compared. IWC had the largest share of jobs that ran with a different value (11%). After a blocked request, the run later completed a job of the same tool in 71% of the 1,700 cases, with matching parameters in 52%. A match shows that Galaxy ran what was asked, not that the settings were scientifically appropriate.

Fig. 3f | Which infrastructure changes could prevent the most failed Galaxy requests?

f Failed requests and candidate infrastructure improvements

Failure classes grouped by the change most likely to help (codebook in Source Data); an independent rater reproduced 92% of the classes and named the same improvement for 89% of requests



Panel f of result_evaluation/figures/fig3.pdf · image: result_evaluation/panels/fig3f.png

RATIONALE

- **Data.** 7,141 failed calls to the Galaxy agent interface, from the agent traces of the four model configurations; 1,339 Galaxy runs had at least one. Each failure has one of 15 classes from the archive's ordered, rule-based classifier. The unit is the failed request; runs are also counted per group.
- **Variables.** Classes are grouped by the change most likely to prevent them (codebook in Source Data): six attributable groups (API design, error diagnostics, tool and parameter descriptions, datatypes and uploads, server capacity, UDT support) and two that cannot be attributed (tool runtime errors; other or unclassified). Bars give each group's requests and share of all failed requests; the right-hand column gives the runs with at least one failure in the group.
- **Analysis.** Descriptive only, with an unvalidated codebook. An independent AI rater, blind to the rule, recoded 150 requests (10 per class; Extended Data Fig. 6d), which is an AI second rating, not human validation.
- **Reading the plot.** Blue bars, attributable groups sorted by size. Grey bars below the rule, groups that cannot be attributed. Labels give each group's count and percentage.

CONCLUSION

API design was the largest attributable group (1,580 requests, 22%, in 651 runs). Error diagnostics came next (1,340, 19%; jobs that failed with no diagnostic message), then tool and parameter descriptions (1,154, 16%). Datatypes and uploads, server capacity and UDT support each accounted for 3–4%. About a third of failures (32%) could not be attributed: tool runtime errors (1,474, 21%) and other or unclassified (833, 12%). The AI rater gave the same class to 92% of sampled requests in rule-defined classes. For 89% (107 of 120) of the sampled requests whose class has an improvement group, that group was among those the rater chose; this is the share the subtitle reports. The panel ranks candidate changes; it does not show that any change would prevent these failures.

Figure 4 | Answer agreement and tool use vary across model configurations

This figure asks whether the four model configurations differ in what they ran in Galaxy and in how consistently their replicate runs answered, and whether consistent answers, consistent tool sets or self-checking go with correct answers. These run-to-run properties matter for reproducibility and auditability, which mean accuracy (Fig. 2) does not capture.

PANELS IN THIS FIGURE

- **Fig. 4a** Which kinds of Galaxy tools did each model run on each benchmark?
- **Fig. 4b** How often did three replicate runs give the same answer, and do models differ?
- **Fig. 4c** Are tasks whose replicate runs used the same Galaxy tools solved more often?
- **Fig. 4d** Did correct runs check their own analyses more often than incorrect runs?
- **Fig. 4e** When replicate runs fail, do they repeat the same rejected answer, and does that track task difficulty?

Fig. 4a | Which kinds of Galaxy tools did each model run on each benchmark?

a What each model ran in Galaxy

Galaxy runs with at least one completed job in each family (%); UDTs were not offered on IWC

		BixBench				CompBio				IWC			
		GPT-5.5	Sol	Luna	DeepSeek	GPT-5.5	Sol	Luna	DeepSeek	GPT-5.5	Sol	Luna	DeepSeek
Data handling	Tables and text	10	38	51	33	17	28	43	33	0	30	30	33
	Format conversion and upload	19	42	57	23	10	16	31	22	10	27	23	10
	Inspection and quality control	2	2	2	0	10	22	26	21	0	17	20	10
	Data retrieval	0	0	0	0	0	2	6	4	10	13	10	13
Scientific methods	Read processing and alignment	2	2	2	2	8	16	18	15	27	40	33	30
	Variant calling and annotation	0	0	0	0	3	6	10	7	0	0	0	3
	Expression and differential testing	3	6	9	5	2	3	4	5	20	17	20	10
	Single-cell and spatial analysis	0	0	0	0	1	4	5	7	3	3	0	10
	Genomic intervals and sequence features	2	2	2	2	8	15	26	23	3	10	7	13
	Phylogenetics	12	16	21	17	0	0	0	0	0	0	0	0
	Sequence search, taxonomy and assembly	0	1	1	1	4	5	8	7	10	27	27	10
	Peak calling and epigenomics	0	0	0	0	1	1	2	2	3	10	10	3
	Statistics, machine learning and enrichment	22	35	34	19	2	4	9	9	0	0	0	0
	Other methods	0	0	0	0	0	0	0	0	3	10	7	10
	UDT (agent-written code)	66	35	39	9	56	75	41	35	0	0	0	0

Panel a of result_evaluation/figures/fig4.pdf · image: result_evaluation/panels/fig4a.png

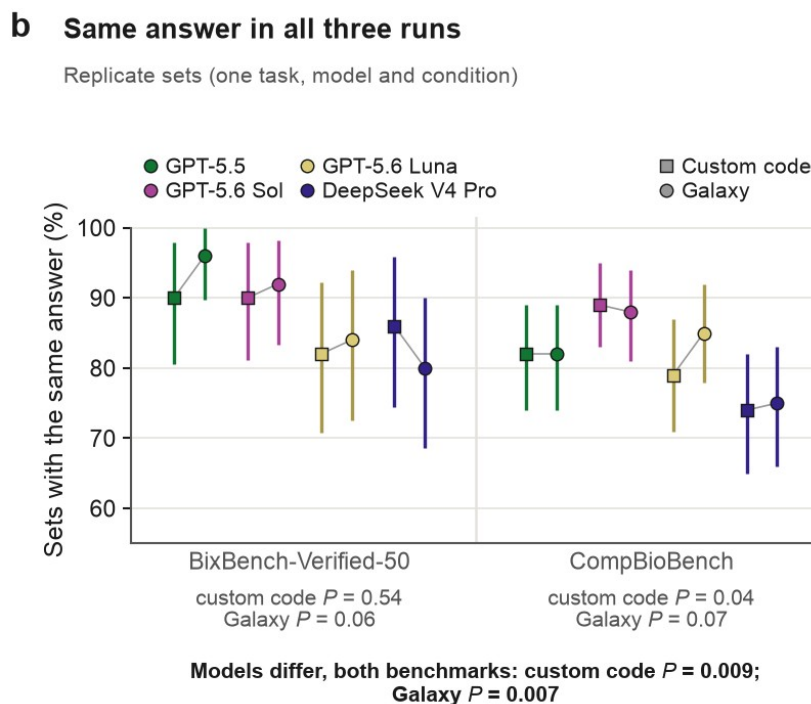
RATIONALE

- **Data.** All 1,908 traced Galaxy-condition runs of the four models: 150 BixBench-Verified-50, 290–300 CompBioBench and 30 IWC runs per model. A tool counts when one of its jobs reached the completed (ok) state in the Galaxy job records captured in the agent trace. The unit is the run.
- **Variables.** The 397 installed tools with a completed job are assigned to 14 method families by ordered rules on the Tool Shed identifier (first match wins; codebook in Source Data): four data-handling and ten scientific-method families. Any completed UDT job (agent-written code run as a Galaxy job) forms its own row. Each cell is the percentage of a model's traced runs on that benchmark with at least one completed job in the family.
- **Analysis.** Descriptive only; no intervals or tests.
- **Reading the plot.** Columns are models within benchmarks; darker cells mean more runs. A run counts in every family it used, so columns do not sum to 100. UDTs were not offered on IWC, so that row is zero by design.

CONCLUSION

Tool use differed by benchmark and by model. Tables-and-text or format-conversion tools were the most used installed tools for three of four models on BixBench-Verified-50 and for all four on CompBioBench. Among scientific families, statistics (19–35% of runs) and phylogenetics (12–21%) led on BixBench-Verified-50, and read processing and alignment (27–40%) on IWC. Reliance on UDTs varied widely: GPT-5.5 ran one in 66% of BixBench-Verified-50 runs, DeepSeek V4 Pro in 9%. The panel shows whether a family was used, not how much, how well, or what UDTs computed (Extended Data Fig. 4e).

Fig. 4b | How often did three replicate runs give the same answer, and do models differ?



Panel b of result_evaluation/figures/fig4.pdf · image: result_evaluation/panels/fig4b.png

RATIONALE

- **Data.** Submitted answers and grades of all BixBench-Verified-50 and CompBioBench runs, grouped into 1,200 replicate sets (three runs of one task by one model in one condition): 600 per condition, 50 or 100 sets per point.
- **Variables.** A set agrees when all three answers match under the task-aware rule: numbers within the benchmark verifier's tolerance (otherwise 0.1%), lists compared as sets, case and spacing ignored. A missing submission counts as disagreement. The y-axis is the percentage of agreeing sets; colour is model, squares are custom code and circles Galaxy.
- **Analysis.** 95% percentile cluster-bootstrap intervals (20,000 resamples; clusters are BixBench source capsules, otherwise tasks). Model effects are tested by permuting model labels within tasks (20,000 permutations). Primary tests pool both benchmarks within each condition (Holm over two); per-benchmark tests are secondary (Holm over four).
- **Reading the plot.** Thin grey lines join each model's two conditions. Per-benchmark P values sit under each facet; the bold line gives the primary tests. No test compares the conditions.

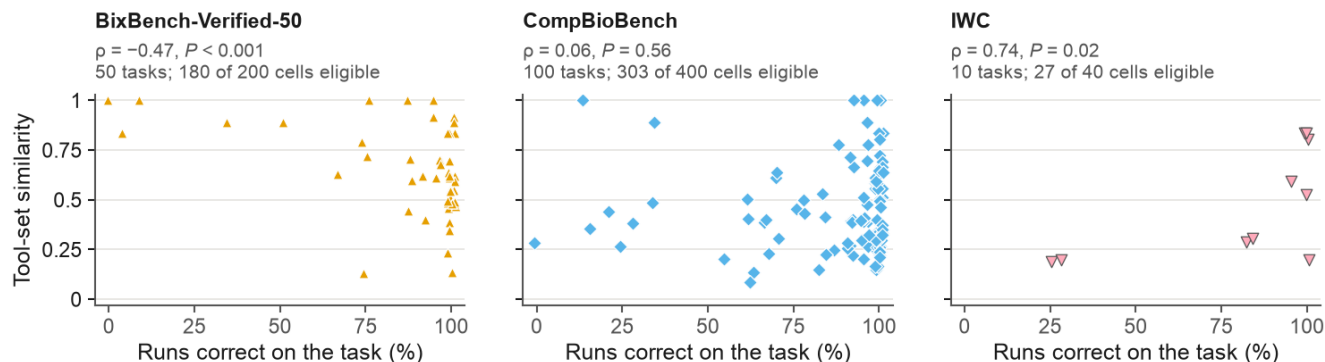
CONCLUSION

Agreement was high but differed between models: 80–96% of sets on BixBench-Verified-50 and 74–89% on CompBioBench. Pooling both benchmarks, a model effect was detected in both conditions (custom code $P = 0.009$, Galaxy $P = 0.007$, Holm-adjusted). Per benchmark, only CompBioBench custom code stayed below 0.05 after adjustment ($P = 0.04$), where GPT-5.6 Sol agreed in 89% of sets (95% interval 83–95%) and DeepSeek V4 Pro in 74% (65–82%). Agreement is not correctness: sets that repeat the same rejected answer count as agreeing (Fig. 4e).

Fig. 4c | Are tasks whose replicate runs used the same Galaxy tools solved more often?

C Tool-set similarity and task accuracy

Galaxy; 1 = the same set of tools in all three runs (any UDT counted as one item; order, versions and parameters ignored). Models compared on the same task: Spearman $\rho = 0.03$, $P = 0.57$



Panel c of result_evaluation/figures/fig4.pdf · image: result_evaluation/panels/fig4c.png

RATIONALE

- **Data.** Galaxy task–model cells in which all three replicate runs completed at least one job: 180 of 200 on BixBench-Verified-50, 303 of 400 on CompBioBench and 27 of 40 on IWC (10 tasks, including host-read removal). Tool sets come from completed jobs in the agent traces. Accuracy uses the results-site grades, which regrade bix-53-q2 (75% of runs correct) and bix-43-q2 (50%).
- **Variables.** A run's tool set is its installed tools (identifier without version) plus one item for any UDT. Tool-set similarity is the mean pairwise Jaccard index (shared ÷ combined tools) of the three runs, averaged over a task's eligible cells. The x-axis is the share of the task's 24 runs (four models × two conditions × three replicates) that were correct; IWC runs count as correct at ≥ 0.99 output agreement.
- **Analysis.** Spearman ρ across tasks within each benchmark, with permutation P values (20,000 permutations); correlations are descriptive. The within-task comparison correlates task-centred similarity with task-centred Galaxy accuracy over 507 cells, permuting within tasks.
- **Reading the plot.** One point per task, jittered horizontally by up to 1.2 points; 1 means the same tool set in all three runs.

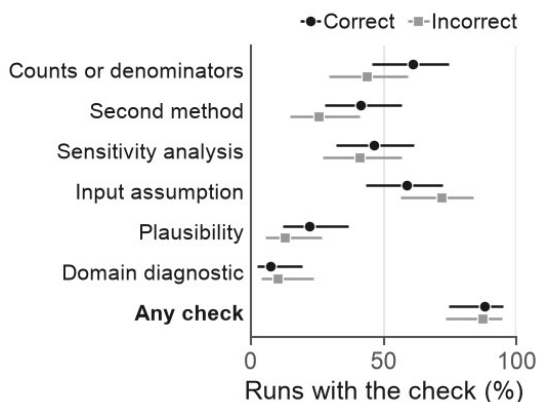
CONCLUSION

The direction of the relation changed with the benchmark. Similarity was negatively correlated with accuracy on BixBench-Verified-50 ($\rho = -0.47$, $P < 0.001$; 50 tasks), no association was detected on CompBioBench ($\rho = 0.06$, $P = 0.56$; 100 tasks), and the correlation was positive on IWC ($\rho = 0.74$, $P = 0.02$; only 10 tasks), although host-read removal was solved by all 24 runs at low similarity (0.20). Comparing models on the same task, no association was detected ($\rho = 0.03$, $P = 0.57$). Consistent tool choice is therefore not a general marker of correct answers. The measure ignores order, versions and parameters (Extended Data Fig. 4b).

Fig. 4d | Did correct runs check their own analyses more often than incorrect runs?

d Verification checks

80 coded runs (coder blind to the grade); 95% intervals



Panel d of result_evaluation/figures/fig4.pdf · image: result_evaluation/panels/fig4d.png

RATIONALE

- **Data.** 80 BixBench-Verified-50 and CompBioBench runs, sampled at 10 per benchmark × condition × outcome, at most one run per task per stratum; DeepSeek V4 Pro supplied 29. AI coders read condensed transcripts blind to the grade; the condition was visible. Outcome is the current results-site grade: one run sampled as incorrect (D079, bix-53-q2) is correct after the regrade, so the groups hold 41 correct and 39 incorrect runs.
- **Variables.** Six check types coded present or absent from a fixed codebook: counts or denominators, second method (the same quantity recomputed by another tool or code path), sensitivity analysis, input assumption, plausibility and domain diagnostic; "any check" means at least one. One coded plausibility check, a download of the public answer file, was removed.
- **Analysis.** Percentages of runs with 95% Wilson intervals, 41 correct and 39 incorrect runs. Descriptive only; no test.
- **Reading the plot.** Black circles are correct runs, grey squares incorrect runs, horizontal bars intervals. Extended Data Fig. 7a splits the same codes by condition.

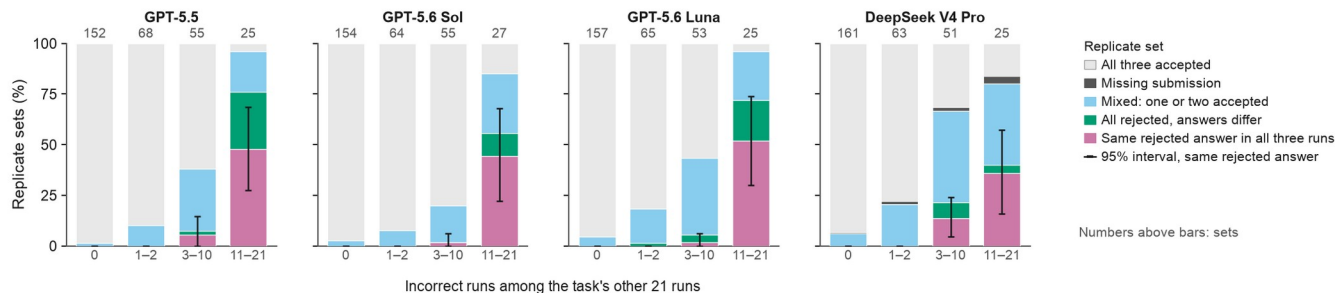
CONCLUSION

Checks were common whatever the outcome: 87.8% of correct and 87.2% of incorrect runs had at least one (95% intervals 74–95% and 73–94%). Correct runs more often checked counts or denominators (61.0% vs 43.6%) and recomputed a result by a second method (41.5% vs 25.6%); incorrect runs more often checked input assumptions (71.8% vs 58.5%). Every pair of intervals overlaps and no test was run. The codes are AI-assisted, not human-validated, and sampling by outcome means these are not population rates.

Fig. 4e | When replicate runs fail, do they repeat the same rejected answer, and does that track task difficulty?

e Replicate outcomes by held-out task difficulty

BixBench-Verified-50 and CompBioBench, both conditions; difficulty from the task's other 21 runs, so a set's own runs never define it



RATIONALE

- **Data.** All 1,200 BixBench-Verified-50 and CompBioBench replicate sets from both conditions, 300 per model, with each run's grade and submitted answer. Grades are the results site's; the bix-53-q2 and bix-43-q2 regrades move nine sets from the 11–21 bin to 3–10.
- **Variables.** Held-out difficulty is the number of incorrect runs among the task's other 21 runs, binned 0, 1–2, 3–10 and 11–21, so a set's own runs never define it. Each set has one outcome: all three accepted, missing submission, mixed (one or two accepted), all rejected with different answers, or the same rejected answer in all three runs (task-aware matching, as in Fig. 4b).
- **Analysis.** Outcome shares per model and bin. Error bars are 95% percentile cluster-bootstrap intervals (20,000 resamples; clusters are BixBench source capsules, otherwise tasks) for the same-rejected-answer share. Descriptive only; no test.
- **Reading the plot.** Pink segments are repeated rejected answers; numbers above bars are sets. The 0 bin is mostly tasks solved by all 24 runs (600 of its 624 sets were all accepted).

CONCLUSION

Repeated rejected answers appeared only on harder tasks. No set repeated a rejected answer when two or fewer of the other 21 runs were incorrect (0 of 884 sets); the share was 5.6% (95% interval 2.5–8.9%; 214 sets) at 3–10 and 45.1% (28.1–63.0%; 102 sets) at 11–21. In the hardest bin every model showed it, from 36% (DeepSeek V4 Pro, 16–57%) to 52% (GPT-5.6 Luna, 30–74%). A repeated rejection can reflect a shared error, an ambiguous reference or the scorer, so on its own it does not measure rigor.

Figure 5 | Galaxy records analyses as structured provenance and uses more input tokens on question-answering tasks

This figure asks what running analyses through Galaxy costs in model tokens relative to custom code, where the extra input comes from, whether interface changes reduce it, and what Galaxy's retained record gives a reviewer in return. The answer matters because the case for Galaxy rests on auditability, and that benefit has to be weighed against the extra context the interface consumes.

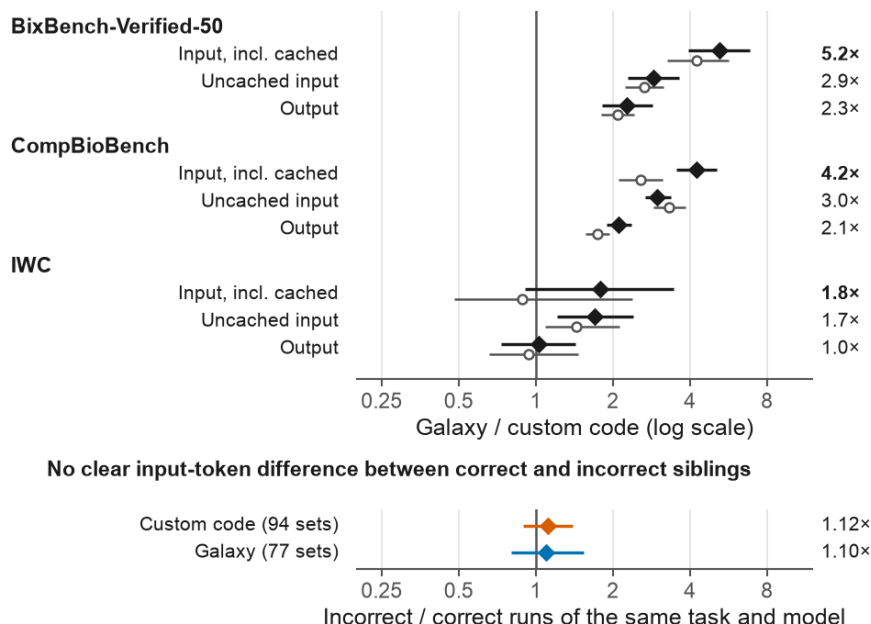
PANELS IN THIS FIGURE

- **Fig. 5a** Does Galaxy use more tokens than custom code on each benchmark, and do incorrect runs use more input than correct ones?
- **Fig. 5b** Does Galaxy's extra input come from more actions or from more input per action, and what does Galaxy return to the agent?
- **Fig. 5c** Can changes to the Galaxy interface reduce Galaxy's token use relative to custom code?
- **Fig. 5d** What does the retained record hold for each analysis step in each condition?
- **Fig. 5e** How does the same analysis step appear in the custom-code trace and in the Galaxy job record?

Fig. 5a | Does Galaxy use more tokens than custom code on each benchmark, and do incorrect runs use more input than correct ones?

a Token use, Galaxy relative to custom code

Diamonds, typical paired task (geometric mean of task–model ratios; primary); circles, all tokens summed over the same tasks (aggregate)



Panel a of result_evaluation/figures/fig5.pdf · image: result_evaluation/panels/fig5a.png

RATIONALE

- **Data.** Scored primary runs of four model configurations (three per task and condition), with per-run token counts from the agent traces, paired into task–model cells: 200 on BixBench-Verified-50, 400 on CompBioBench and 40 on IWC (10 tasks, including host-read removal). Below, replicate sets (three runs of one task, model and condition) with both outcomes under the results-site grades: 94 custom-code and 77 Galaxy sets, models and benchmarks pooled.
- **Variables.** Galaxy / custom-code ratios of input tokens (including cached context), uncached input (input minus cached) and output tokens. Below, input tokens of incorrect relative to correct runs; a run is correct when accepted or, for IWC, at ≥ 0.99 output agreement.
- **Analysis.** Primary: geometric mean over cells of the ratio of median tokens per run; aggregate: ratio of total tokens over the same cells; below, geometric mean over sets of median incorrect / median correct input. Intervals: 95% percentile cluster-bootstrap (20,000 resamples; clusters are BixBench source capsules, otherwise tasks). Holm-adjusted P values from paired cluster sign-flip tests (200,000 draws; exact for IWC) are in Source Data.
- **Reading the plot.** Log x-axis; 1 means equal use. Filled diamonds, primary estimates (printed at right, input in bold); open circles, aggregate; below, colour marks the condition.

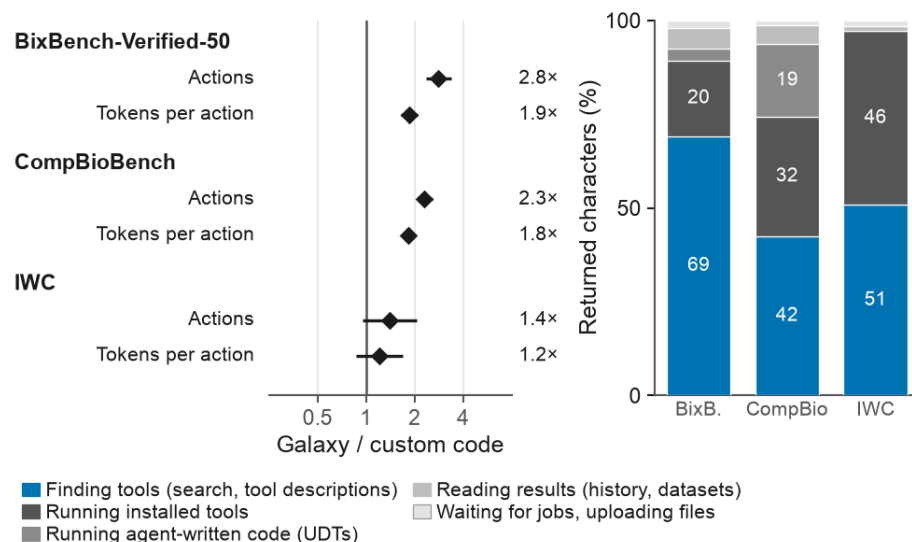
CONCLUSION

On the question-answering benchmarks, Galaxy used clearly more tokens. Input was 5.2× (95% interval 3.9–6.8) on BixBench-Verified-50 and 4.2× (3.5–5.1) on CompBioBench, with uncached input at 2.9× and 3.0× and output at 2.3× and 2.1× (all $P < 0.001$). Aggregate input ratios were lower (4.3× and 2.6×). On IWC, Galaxy did not use clearly more input (1.8×, 0.9–3.4; aggregate 0.89×); uncached input was higher (1.7×, 1.2–2.4), although its Holm-adjusted P was 0.08. Incorrect runs used about as much input as correct runs of the same task and model (1.12× in custom code, 0.89–1.39; 1.10× in Galaxy, 0.80–1.54). Token ratios are not monetary costs.

Fig. 5b | Does Galaxy's extra input come from more actions or from more input per action, and what does Galaxy return to the agent?

b Where the extra input comes from

Left, Galaxy / custom code, typical paired task (all runs); right, the text Galaxy returned to the agent, by what the request was for



Tools inspected but not executed in the same run: 45–53% by benchmark.

Median share of input that was cached (earlier context reread): Galaxy 96%, custom code 93–96%.

Panel b of result_evaluation/figures/fig5.pdf · image: result_evaluation/panels/fig5b.png

RATIONALE

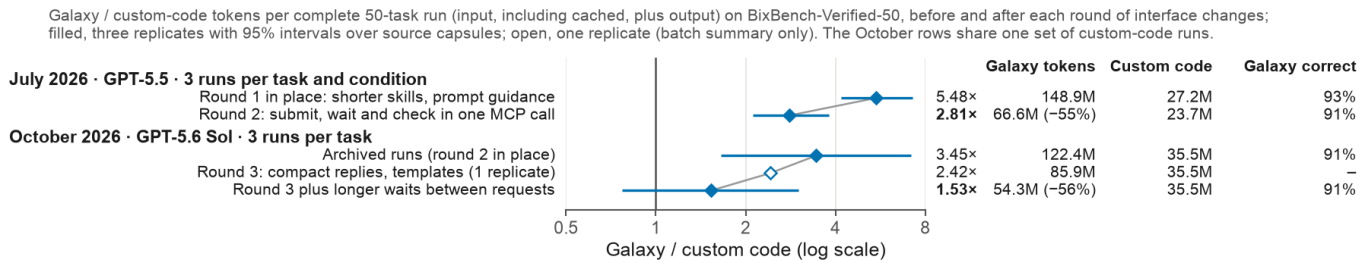
- **Data.** Left, the task–model cells of panel a (200, 400 and 40), with each run's action count from the archive's per-run action table. Right, characters returned by Galaxy interface calls in 1,908 traced Galaxy runs. The notes use the inspected-tool records (1,942, 6,357 and 788 inspected tools by benchmark) and per-run cached-token counts.
- **Variables.** Actions are the agent's tool calls: shell commands, Galaxy interface calls, web searches or fetches, and file reads, writes and edits. Tokens per action is input tokens (including cached context) divided by actions. Bars give each request type's share of all characters Galaxy returned in a benchmark.
- **Analysis.** Left, the primary estimator of panel a, with 95% cluster-bootstrap intervals; sign-flip *P* values and a correct-runs-only sensitivity analysis are in Source Data. Right and notes, descriptive only.
- **Reading the plot.** Left, diamonds on a log axis, where 1 means equal. Right, stacked bars sum to 100% per benchmark; blue marks finding tools, and segments under 9% are unlabelled.

CONCLUSION

On the question-answering benchmarks, Galaxy's extra input reflects both more actions (2.8×, 95% interval 2.4–3.4, on BixBench-Verified-50; 2.3×, 2.1–2.6, on CompBioBench) and more input per action (1.9×, 1.6–2.1; 1.8×, 1.7–2.0). On IWC neither ratio is clearly above 1 (1.4×, 0.96–2.05; 1.2×, 0.86–1.68). Finding tools produced the largest share of returned characters in every benchmark (42–69%), and 45–53% of inspected tools were not executed in the same run. The median cached share was 96% in Galaxy and 93–96% in custom code, so most input is earlier context reread. The split is not causal: because context accumulates, more actions also raise input per action.

Fig. 5c | Can changes to the Galaxy interface reduce Galaxy's token use relative to custom code?

C Reducing Galaxy token use



Panel c of result_evaluation/figures/fig5.pdf · image: result_evaluation/panels/fig5c.png

RATIONALE

- **Data.** Complete 50-task BixBench-Verified-50 runs from the token_improvement records. July (GPT-5.5): the July 6 batch with round 1 in place against the archived runs made after round 2, 150 runs per condition in each, both conditions rerun. October (GPT-5.6 Sol): the archived Galaxy runs, one intermediate replicate after round 3 (batch summary totals only), and 150 new Galaxy runs after round 3 plus longer waits, all against the same archived custom-code runs. Correctness follows the results site (for the July 6 batch, its July page).
- **Variables.** Tokens are input (including cached context) plus output, summed over the 50 tasks and averaged over replicates. The columns give Galaxy and custom-code millions of tokens per 50-task run, the Galaxy change from the first to the last stage, and the share of Galaxy runs correct.
- **Analysis.** Ratio of tokens per 50-task run, with 95% cluster-bootstrap intervals over source capsules (20,000 resamples); the Galaxy change is paired by task and uses the same bootstrap. The one-replicate stage has no interval, and no tests were run.
- **Reading the plot.** Log x-axis. Filled diamonds, three replicates with intervals; open diamond, one replicate. Grey lines join the stages of each comparison; bold values mark each comparison's last stage.

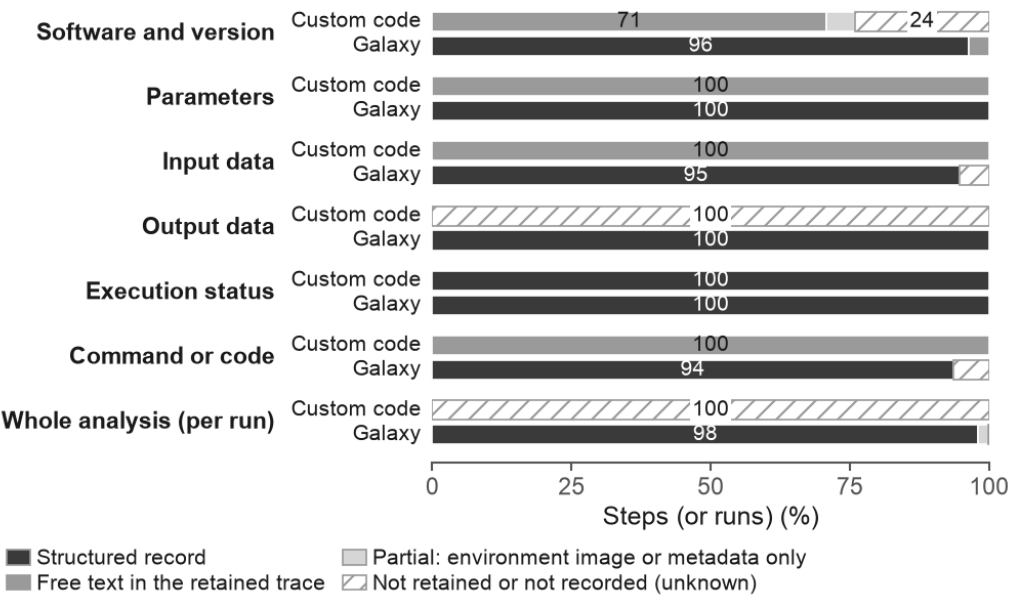
CONCLUSION

In both comparisons, Galaxy token use fell by more than half. In July, the ratio fell from 5.48× (95% interval 4.17–7.26) to 2.81× (2.11–3.80), and Galaxy tokens fell 55% (95% interval 45–63%). In October, it fell from 3.45× (1.65–7.15) through 2.42× (one replicate) to 1.53× (0.77–3.01), and Galaxy tokens fell 56% (46–63%). Galaxy correctness stayed at 91–93% (custom code, 89–90%). These are batch comparisons: several changes were bundled, the October agent CLI version also changed (0.146.0 to 0.156.0) and its custom-code baseline was not rerun. The panel does not estimate the effect of any single change.

Fig. 5d | What does the retained record hold for each analysis step in each condition?

d What the retained record holds for each analysis step

56,404 custom-code steps (shell commands labelled analysis) and 21,918 Galaxy jobs;
whole analysis per run (1,920 runs per condition)



Panel d of result_evaluation/figures/fig5.pdf · image: result_evaluation/panels/fig5d.png

RATIONALE

- **Data.** Per-task evidence files for all primary runs: 21,918 Galaxy jobs and 56,404 custom-code shell commands labelled as analysis. The whole-analysis row covers 1,920 runs per condition. Nothing was rerun.
- **Variables.** Each step's record of six elements (software and version, parameters, input data, output data, execution status, command or code) is classed as structured record, free text in the retained trace, partial (environment image or history metadata only), or not retained (unknown, not absent). A run's whole analysis counts as structured when its Galaxy history was retrieved.
- **Analysis.** Descriptive only: percentages of steps (or runs) per class, assigned by fixed rules. Galaxy elements are structured when the job record holds the field, and UDT software only with a versioned container. Custom-code parameters, inputs and commands count as free text, exit codes as structured, versions as free text when any command in the run printed one, and output files as not retained.
- **Reading the plot.** Each bar sums to 100%. Dark, structured; mid-grey, free text; light grey, partial; hatched, not retained. Segments under 12% are unlabelled.

CONCLUSION

Galaxy kept a structured record of each element for 94–100% of jobs (software and version 96%, input data 95%, command 94%), and 98% of Galaxy runs had a retrieved history. Custom-code steps kept parameters, inputs and code as free text in the trace (100%) and software versions as free text in 71%. Exit codes were structured, but output files and the whole analysis were not retained. The panel compares what the benchmark retained, not what custom code could record, and it does not test reproducibility.

Fig. 5e | How does the same analysis step appear in the custom-code trace and in the Galaxy job record?

e One analysis step, recorded both ways

bix-45-q1, PhyKIT relative composition variability; GPT-5.6 Sol, replicate 1 (the case in Fig. 2d)

	Custom code: retained trace	Galaxy: job record
Software and version	PhyKIT 2.4.1 installed, then 2.0.3 downloaded and loaded through PYTHONPATH (in command text only)	phykit_metrics wrapper 0.2.0+galaxy0 (Tool Shed ID); library version not in the job record
Parameters	In the Python code of the command (file suffix, rounding)	Fields: metric = relative_composition_variability; mode = zip_archive; suffix = .mafft
Inputs and outputs	Input archives named in the command; output files not retained	1 input and 3 output datasets, linked by ID in the history
Status	Exit code 0 for every command	Job state "ok", exit code 0
Answer	Accepted	Rejected: the reference encodes the older PhyKIT value

The Galaxy record keeps fields a reviewer can query; the custom-code trace keeps the code, so the same facts must be read from it. Neither record was rerun to test reproducibility.

Panel e of result_evaluation/figures/fig5.pdf · image: result_evaluation/panels/fig5e.png

RATIONALE

- **Data.** One BixBench-Verified-50 task, bix-45-q1 (PhyKIT relative composition variability), with GPT-5.6 Sol replicate 1 in each condition; this is the case in Fig. 2d. Entries come from the task's retained evidence file: the PhyKIT Galaxy job, and the custom-code run's shell commands labelled as analysis. Outcomes come from the graded runs.
- **Variables.** Five rows: software and version, parameters, inputs and outputs, status, and the graded answer. Galaxy entries are job-record fields (tool ID, parameter fields, dataset IDs, job state, exit code). Custom-code entries are what the command text and exit codes show.
- **Analysis.** Descriptive single case; no statistics. The script reads each entry from the evidence and stops if key facts change (the pinned PhyKIT version, its loading through PYTHONPATH, no retained custom-code outputs).
- **Reading the plot.** Read across a row to compare the two records. The orange column is the custom-code trace; the blue column is the Galaxy job record.

CONCLUSION

The Galaxy job record held the wrapper and its version (phykit_metrics 0.2.0+galaxy0), three parameter fields, links to 1 input and 3 output datasets, and the job state as queryable fields. The custom-code trace held the same facts only inside command text, and its output files were not retained. Only the custom-code trace recorded the PhyKIT library version (2.0.3, loaded through PYTHONPATH); the Galaxy job record does not hold it. The Galaxy answer was rejected because the reference encodes the older PhyKIT value. This is one illustrative case, and neither record was rerun.

Extended Data Fig. 2 | Failure causes and sensitivity of the accuracy comparison

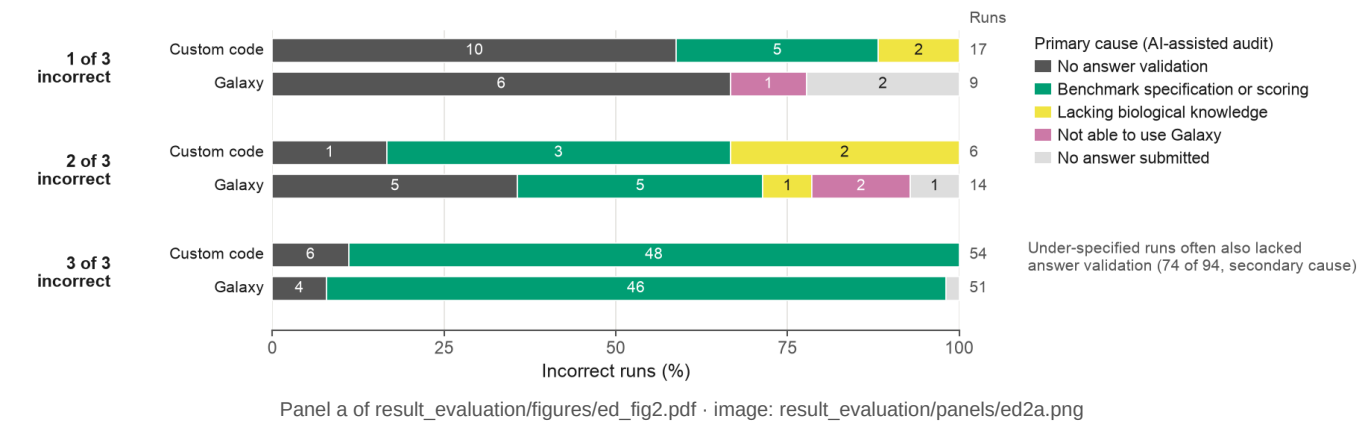
This figure checks two things that Fig. 2 condenses. It gives the full census of causes for incorrect BixBench-Verified-50 runs, and it asks whether the Galaxy minus custom-code difference holds under other correctness thresholds and run populations. The conclusion of Fig. 2 should not depend on one IWC threshold or on a few problematic tasks.

PANELS IN THIS FIGURE

- **Extended Data Fig. 2a** What caused each incorrect BixBench-Verified-50 run, by how many runs of its replicate set failed?
- **Extended Data Fig. 2b** Does the Galaxy minus custom-code difference change with the IWC correctness threshold or the population analysed?

Extended Data Fig. 2a | What caused each incorrect BixBench-Verified-50 run, by how many runs of its replicate set failed?

a Primary cause of every incorrect run, by incorrect runs in its set (BixBench-Verified-50)



RATIONALE

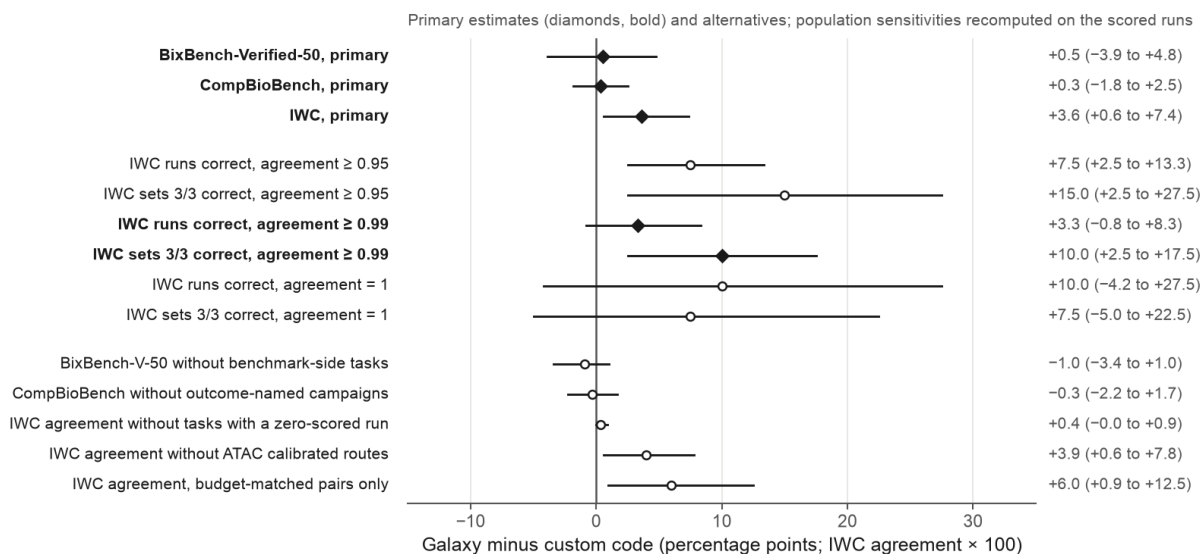
- **Data.** All 151 BixBench-Verified-50 runs graded incorrect on the results site (77 custom code, 74 Galaxy), with the primary cause assigned by the AI-assisted run-level audit; 4 bix-43-q2 runs made incorrect only by the site's regrade take the task-level audit's cause (benchmark specification or scoring). These are the same records as Fig. 2d, but rows are not split by pair type, so concordant and discordant pairs are pooled.
- **Variables.** Rows group runs by condition and by the number of incorrect runs in their replicate set (1, 2 or 3 of 3). Bars give each primary cause's share of the row; segment labels are run counts, and the column at right gives the row total. The note counts runs coded as under-specified, a subset of the green group, that also had missing answer validation as the secondary cause.
- **Analysis.** Descriptive counts and shares; no test.
- **Reading the plot.** Colours follow the five cause groups of Fig. 2d, without hatching. The unlabelled sliver in the Galaxy 3-of-3 row is one run with no answer submitted.

CONCLUSION

Failures repeated in all three runs dominate, and the benchmark is the main attributed cause: 54 custom-code and 51 Galaxy runs come from sets that failed 3 of 3, and 48 and 46 of these were coded as benchmark specification or scoring. Isolated failures (1 of 3) were mostly missing answer validation (10 of 17 custom-code runs, 6 of 9 Galaxy runs). Not able to use Galaxy was the primary cause of 3 of 74 incorrect Galaxy runs. Most under-specified runs also lacked answer validation (74 of 94), so these two causes are not cleanly separable.

Extended Data Fig. 2b | Does the Galaxy minus custom-code difference change with the IWC correctness threshold or the population analysed?

b Sensitivity of the condition difference



Panel b of result_evaluation/figures/ed_fig2.pdf · image: result_evaluation/panels/ed2b.png

RATIONALE

- **Data.** The pooled primary estimates of Fig. 2b; the 240 IWC runs and 80 IWC replicate sets (host-read removal included) re-scored at three agreement thresholds; and five population sensitivities, each dropping tasks or runs, recomputed on these scores with the primary estimator rather than read from the archive's accuracy_sensitivities.csv, which used its own grades and nine IWC tasks.
- **Variables.** The x axis is Galaxy minus custom code in percentage points (IWC agreement $\times 100$). Threshold rows give runs correct and sets with 3/3 correct at agreement ≥ 0.95 , ≥ 0.99 and = 1. Population rows drop BixBench-Verified-50 tasks whose failures the audit attributes to the benchmark (22 capsules left), CompBioBench campaigns whose names reference target scores or wrong answers, IWC tasks with a zero-scored run (7 tasks left) or agent-calibrated ATAC routes (9 left), or keep only IWC pairs with matching time budgets.
- **Analysis.** Estimators and 95% percentile cluster-bootstrap intervals are as in Fig. 2b. Threshold and population rows have unadjusted sign-flip P values in Source Data (exact for IWC). With ten or fewer IWC clusters the intervals are too narrow.
- **Reading the plot.** Bold diamonds mark the primary estimates and the ≥ 0.99 rows used in Fig. 2; open circles are alternatives. Lines are 95% intervals, and the text at right repeats each estimate and interval.

CONCLUSION

The BixBench-Verified-50 and CompBioBench differences stay near zero when the flagged tasks or campaigns are dropped (-1.0 , -3.4 to $+1.0$, $P = 0.63$; -0.3 , -2.2 to $+1.7$, $P = 0.84$). Every IWC point estimate favours Galaxy, but its size depends on the definition: from $+0.4$ (-0.0 to $+0.9$; $P = 0.44$) without the three tasks with a zero-scored run to $+15.0$ ($+2.5$ to $+27.5$) for sets correct at ≥ 0.95 . No IWC row reached $P < 0.05$ (primary, 0.055; smallest alternative, 0.06 for runs at ≥ 0.95 and for budget-matched pairs). The IWC advantage therefore rests on a few tasks and is not established.

Extended Data Fig. 3 | Task status, execution errors and final correctness

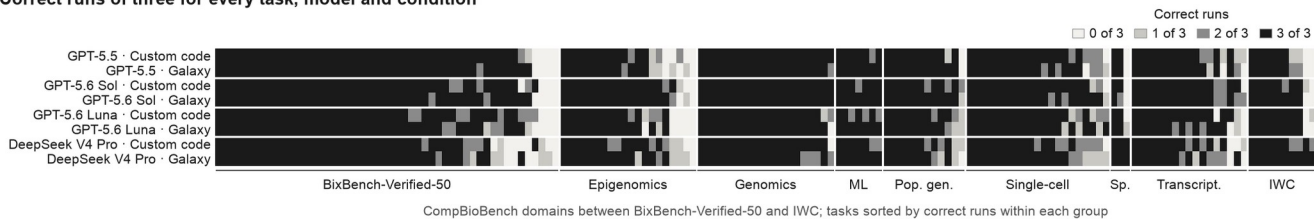
This figure gives the detail behind Fig. 3a, c and d: correctness for every task, model configuration and condition; all seven error types in each channel; and the error-bin analysis of Fig. 3d within each benchmark. It shows whether the pooled results depend on a few tasks or on one benchmark.

PANELS IN THIS FIGURE

- **Extended Data Fig. 3a** Are failures spread across tasks, or concentrated in a few tasks that fail in both conditions?
- **Extended Data Fig. 3b** Which kinds of execution error occurred in each channel?
- **Extended Data Fig. 3c** Does the relation between execution errors and correctness in Fig. 3d hold within each benchmark?

Extended Data Fig. 3a | Are failures spread across tasks, or concentrated in a few tasks that fail in both conditions?

a Correct runs of three for every task, model and condition



Panel a of result_evaluation/figures/ed_fig3.pdf · image: result_evaluation/panels/ed3a.png

RATIONALE

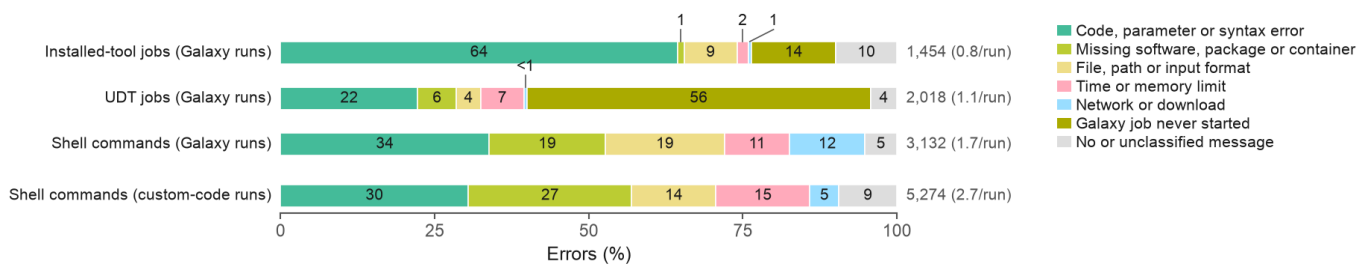
- **Data.** All 3,840 scored primary runs on 160 tasks (50 BixBench-Verified-50, 100 CompBioBench, 10 IWC), graded as in Fig. 3a (BixBench-Verified-50 as displayed on the results site; IWC including host-read removal). The unit is a cell of three replicate runs for one task, model configuration and condition: 1,280 cells.
- **Variables.** Columns are tasks, grouped as BixBench-Verified-50, then the CompBioBench domains, then IWC, and sorted within each group by mean correct runs across cells. Rows are the four model configurations, each in custom code and in Galaxy. The shade of a cell gives its correct runs of three, with correctness defined as in Fig. 3a.
- **Analysis.** Descriptive only.
- **Reading the plot.** Black, 3 of 3; dark grey, 2 of 3; light grey, 1 of 3; near-white, 0 of 3. Each model's custom-code row sits directly above its Galaxy row, so the two conditions can be compared on the same task.

CONCLUSION

Most cells were fully correct (1,016 of 1,280 at 3 of 3), and 80 of 160 tasks were correct in all 24 runs. Failures clustered in a minority of tasks and were usually shared by both conditions. Of the 15 tasks with at most half of their Galaxy runs correct, 13 also had at most half of their custom-code runs correct. Two tasks were never correct. A few tasks differed sharply between conditions in either direction: bix-45-q1 was correct in 8 of 12 custom-code runs and 0 of 12 Galaxy runs, and bix-30-q3 in 4 of 12 and 12 of 12. The panel shows where failures fall, not why.

Extended Data Fig. 3b | Which kinds of execution error occurred in each channel?

b Execution errors by type and channel (all seven types)



Panel b of result_evaluation/figures/ed_fig3.pdf · image: result_evaluation/panels/ed3b.png

RATIONALE

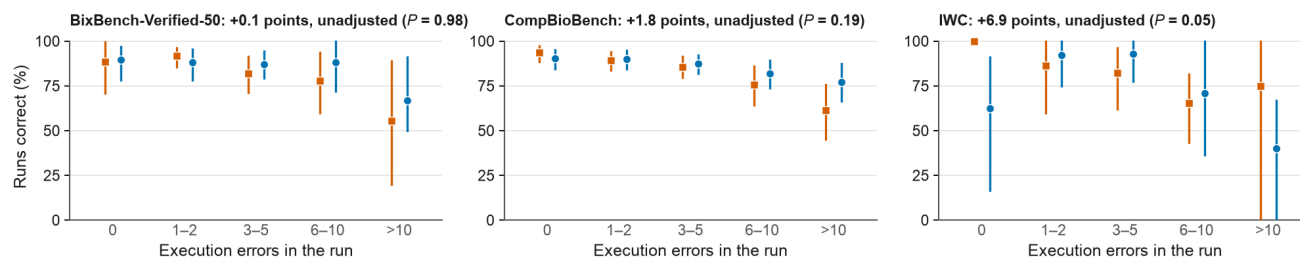
- **Data.** All 11,878 execution errors in the 3,791 scored runs with execution records (1,871 Galaxy, 1,920 custom code): the archive's per-error table plus 71 errors in the 24 IWC host-read removal runs, extracted with the archive's own rules. Failed shell commands come from the agent traces, and Galaxy jobs in the error state come from the archived job records. A job error counts as a UDT error when its tool matches a UDT identifier in the interface calls.
- **Variables.** Ordered rules assign each error one of seven types from its message, exit code and command; "Galaxy job never started" is a job with neither a command line nor an output, and Fig. 3c merges the seven types into four. The right-hand column gives each channel's errors and errors per run (errors divided by the condition's runs).
- **Analysis.** Descriptive only: counts and shares.
- **Reading the plot.** One 100% bar per channel. Numbers are percentages of that channel's errors. Small segments are labelled above the bar, with "<1" for shares below 0.5%.

CONCLUSION

Error types differed by channel. Installed-tool job errors were mostly code, parameter or syntax errors (64%), whereas UDT job errors were mostly jobs that never started (56%). Shell commands in custom-code runs had more missing-software errors (27% against 19%) and more time or memory limits (15% against 11%) than shell commands in Galaxy runs. Galaxy-run shell commands had more network or download errors (12% against 5%). Per run, Galaxy runs had 0.8 installed-tool, 1.1 UDT and 1.7 shell errors; custom-code runs had 2.7 shell errors. The types come from error text, so they describe the reported symptom, not the root cause.

Extended Data Fig. 3c | Does the relation between execution errors and correctness in Fig. 3d hold within each benchmark?

C Final correctness by execution errors, per benchmark



RATIONALE

- **Data.** The same 3,791 scored runs with execution records as in Fig. 3d, split by benchmark. Runs with errors number 858 for BixBench-Verified-50, 1,795 for CompBioBench and 222 for IWC (including host-read removal). The unit is the run.
- **Variables.** x is the number of execution errors in the run, binned as in Fig. 3d. y is the share of runs correct in each bin and condition. Each facet title gives the unadjusted Galaxy minus custom-code difference among runs with errors.
- **Analysis.** Per-bin shares with 95% percentile cluster-bootstrap intervals (20,000 resamples; clusters are BixBench source capsules, otherwise tasks). Bins with fewer than five runs would be omitted, but none had so few. *P* values come from paired cluster randomization tests (200,000 draws) and are not adjusted across the three benchmarks.
- **Reading the plot.** Orange squares, custom code; blue circles, Galaxy; vertical lines, 95% intervals. Each benchmark has its own facet on a 0–100% axis.

CONCLUSION

No benchmark showed a detectable difference among runs with errors: +0.1 points for BixBench-Verified-50 ($P = 0.98$), +1.8 for CompBioBench ($P = 0.19$) and +6.9 for IWC ($P = 0.052$). In CompBioBench, correctness fell with more errors in both conditions, as in the pooled panel. In BixBench-Verified-50, Galaxy runs stayed 87–89% correct up to 10 errors, while custom-code runs fell from 92% (1–2 errors) to 78% (6–10); both were lowest above 10 errors. IWC bins hold 5–42 runs, so their intervals are wide (0–100% for custom code above 10 errors). The pooled result in Fig. 3d is weighted toward CompBioBench, which contributes 1,795 of the 2,875 runs with errors.

Extended Data Fig. 4 | Tool inventory, route similarity, answer matching, difficulty and UDT methods

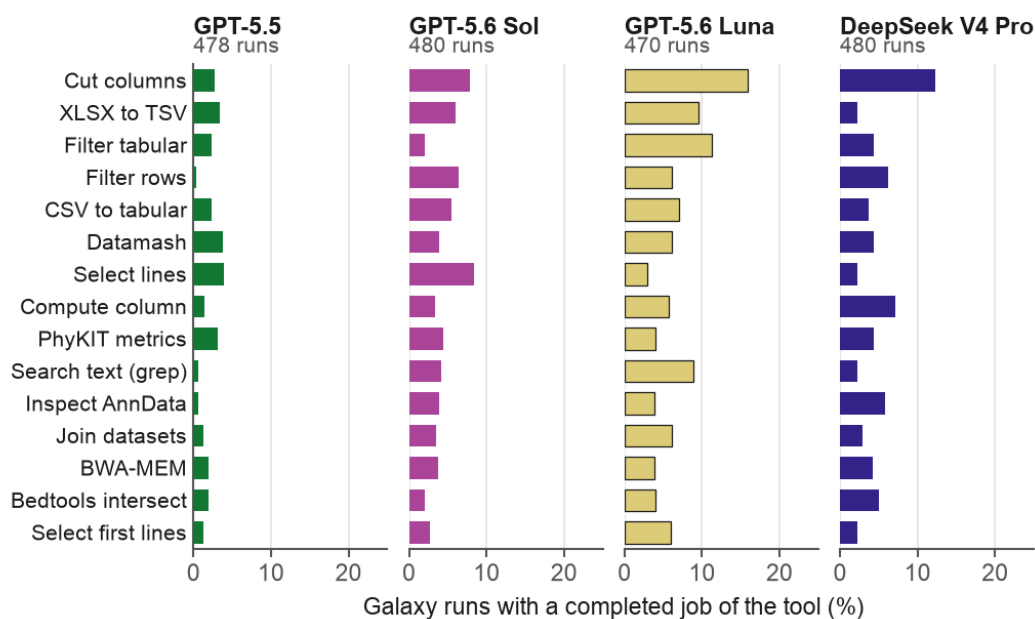
This figure gives the detail behind the Fig. 4 summaries and tests its main analytical choices: how a route is defined, how answers are matched and how difficulty is held out. It also shows what agent-written UDT code computed. Together these show which Fig. 4 results are robust to those choices.

PANELS IN THIS FIGURE

- **Extended Data Fig. 4a** Which installed Galaxy tools were run most often, and by which models?
- **Extended Data Fig. 4b** Do models differ in tool-set similarity, and does the Fig. 4c correlation hold under other route definitions?
- **Extended Data Fig. 4c** Does the answer-matching rule change how often replicate runs agree?
- **Extended Data Fig. 4d** Does the difficulty pattern of Fig. 4e hold when the same model's runs are left out of difficulty?
- **Extended Data Fig. 4e** What did the completed UDT jobs compute?

Extended Data Fig. 4a | Which installed Galaxy tools were run most often, and by which models?

a Installed tools with completed jobs in the most Galaxy runs (15)



Panel a of result_evaluation/figures/ed_fig4a.pdf · image: result_evaluation/panels/ed4a.png

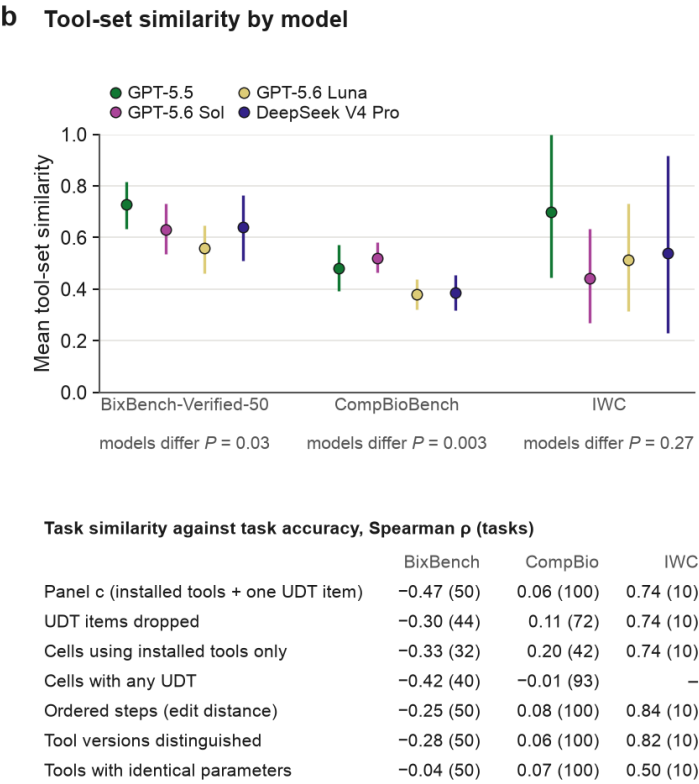
RATIONALE

- **Data.** The 1,908 traced Galaxy runs and completed jobs of Fig. 4a, pooled across the three benchmarks: 478 (GPT-5.5), 480 (GPT-5.6 Sol), 470 (GPT-5.6 Luna) and 480 (DeepSeek V4 Pro) runs.
- **Variables.** Installed tools are identified without version; UDTs are excluded. The 15 tools with completed jobs in the most runs (all models pooled) are shown; ranking counts runs, not jobs, so repeated jobs in one run count once. Bar length is the percentage of each model's traced Galaxy runs with at least one completed job of the tool.
- **Analysis.** Descriptive only; no intervals or tests.
- **Reading the plot.** One panel per model, all sharing the tool order (most-used first) and a 0–25% axis. Tool names are display labels; Source Data gives the Tool Shed identifiers.

CONCLUSION

No installed tool was used widely. The most frequent, Cut columns, completed a job in 2.7–16.0% of a model's runs, and no bar exceeds 16.0%. Twelve of the 15 tools handle tables, text, format conversion or inspection; only PhyKIT metrics, BWA-MEM and Bedtools intersect are scientific-method tools. GPT-5.5 ran each listed tool in at most 4.0% of its runs, in line with its heavier UDT use (Fig. 4a). Installed-tool use was spread across many tools, mostly for data handling.

Extended Data Fig. 4b | Do models differ in tool-set similarity, and does the Fig. 4c correlation hold under other route definitions?



Panel b of result_evaluation/figures/ed_fig4.pdf · image: result_evaluation/panels/ed4b.png

RATIONALE

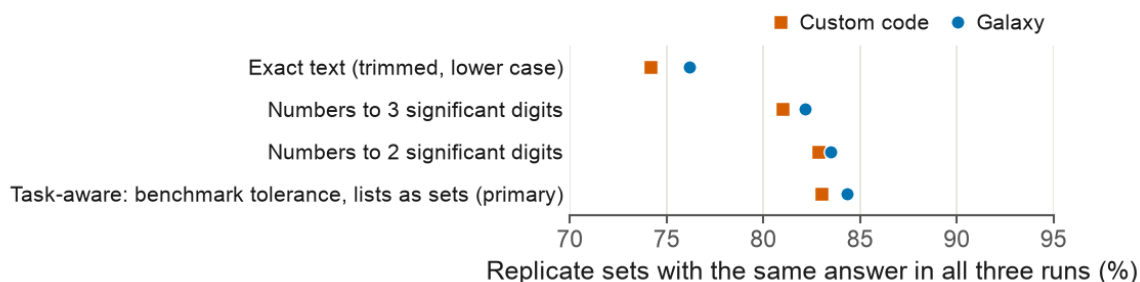
- Data.** The eligible Galaxy task–model cells of Fig. 4c (180 BixBench-Verified-50, 303 CompBioBench, 27 IWC; 4–100 per model and benchmark), with its results-site grades and 10 IWC tasks. The ordered, versioned and parameter variants use the completed analysis jobs in the retained Galaxy history records.
- Variables.** The plot shows mean tool-set similarity per model and benchmark. The table gives Spearman ρ between task similarity and task accuracy (tasks in parentheses) under the Fig. 4c definition and six alternatives: UDT items dropped; cells using installed tools only; cells with any UDT; ordered steps (one minus the normalised edit distance between job sequences); tools distinguished by version; tools with identical non-dataset parameters.
- Analysis.** 95% percentile cluster-bootstrap intervals (20,000 resamples; clusters are BixBench source capsules, otherwise tasks). Model effects are tested by permuting model labels within tasks (20,000 permutations), without adjustment. Table correlations are descriptive.
- Reading the plot.** Points are model means, coloured by model, with vertical lines for 95% intervals; the P value under each benchmark tests the model effect. In the table, each row is one route definition, and a dash marks IWC cells with any UDT, as UDTs were not offered there.

CONCLUSION

Models differed on BixBench-Verified-50 ($P = 0.03$; GPT-5.5 highest at 0.73, 95% interval 0.63–0.82) and CompBioBench ($P = 0.003$; GPT-5.6 Sol 0.52, 0.46–0.58, against 0.38 and 0.39 for GPT-5.6 Luna and DeepSeek V4 Pro); no difference was detected on IWC ($P = 0.27$; 4–9 cells per model). The negative BixBench-Verified-50 correlation weakened when order or versions counted ($\rho = -0.25, -0.28$) and vanished with parameters ($\rho = -0.04$). CompBioBench stayed between -0.01 and 0.20 . IWC stayed positive with order or versions ($\rho = 0.84, 0.82$) and fell to 0.50 with parameters. Eligibility varied, so model means cover different tasks (31 of 50 BixBench-Verified-50 cells for DeepSeek V4 Pro).

Extended Data Fig. 4c | Does the answer-matching rule change how often replicate runs agree?

C Answer agreement under four answer-matching rules



Panel c of result_evaluation/figures/ed_fig4.pdf · image: result_evaluation/panels/ed4c.png

RATIONALE

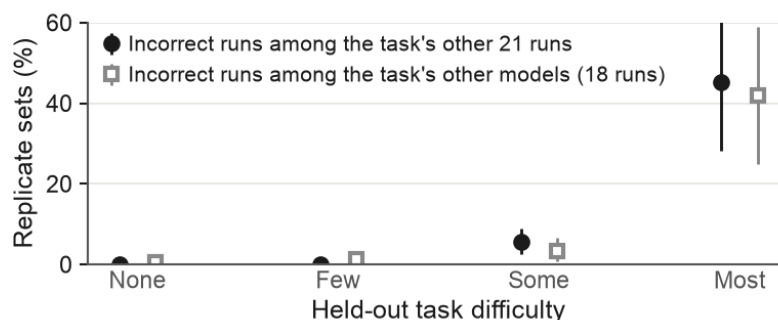
- **Data.** The 1,200 replicate sets of Fig. 4b, 600 per condition (150 tasks × four models).
- **Variables.** The percentage of sets whose three submitted answers match under four rules: exact text after trimming and lower-casing; numbers rounded to three or to two significant digits, which changes only answers that parse as a single number (others stay exact text); and the primary task-aware rule of Fig. 4b (benchmark tolerance or 0.1%, lists as sets, identifier versions ignored). A missing submission never matches.
- **Analysis.** Descriptive only; pooled percentages without intervals or tests.
- **Reading the plot.** Orange squares are custom code and blue circles Galaxy; one row per rule, with the primary rule last. The x-axis starts at 70%, which magnifies small gaps.

CONCLUSION

The rule moved agreement by 8–9 points, more than the gap between conditions. Exact text gave 74.2% (custom code) and 76.2% (Galaxy); three significant digits 81.0% and 82.2%; two significant digits 82.8% and 83.5%; the task-aware rule 83.0% and 84.3%. The gain over exact text comes from numeric answers that differed only in format or in later digits. Galaxy sets agreed slightly more often under every rule (by 0.7–2.0 points), but this comparison has no interval or test.

Extended Data Fig. 4d | Does the difficulty pattern of Fig. 4e hold when the same model's runs are left out of difficulty?

d Same rejected answer by held-out difficulty, two definitions



Panel d of result_evaluation/figures/ed_fig4.pdf · image: result_evaluation/panels/ed4d.png

RATIONALE

- **Data.** The 1,200 BixBench-Verified-50 and CompBioBench replicate sets of Fig. 4e, all models and both conditions pooled, with the results-site grades.
- **Variables.** The share of sets with the same rejected answer in all three runs, by held-out difficulty defined two ways: incorrect runs among the task's other 21 runs (bins 0, 1–2, 3–10, 11–21, as in Fig. 4e) or among the other three models' 18 runs (bins 0, 1–2, 3–8, 9–18). The second removes the same model's runs in the other condition. Bins are labelled None, Few, Some and Most.
- **Analysis.** 95% percentile cluster-bootstrap intervals (20,000 resamples; clusters are BixBench source capsules, otherwise tasks). Descriptive only; no test.
- **Reading the plot.** Filled black circles use the 21-run definition, open grey squares the 18-run definition; bins hold 102–658 sets.

CONCLUSION

Yes. In the hardest bin, 45.1% of sets (95% interval 28.1–63.0%; 102 sets) repeated the same rejected answer under the 21-run definition and 41.8% (25.0–59.0%; 110 sets) under the 18-run definition. In the easiest bin the shares were 0% and 0.5% (0–1.2%). The link between difficulty and repeated rejected answers therefore does not depend on the same model's runs in the other condition. Bin edges differ between the definitions, so the middle bins are not strictly comparable.

Extended Data Fig. 4e | What did the completed UDT jobs compute?

e What completed UDT jobs computed

Share of each model's completed UDT jobs by method class (rules on the UDT definition: code first, then a tool-specific container)

	BixBench				CompBio			
Statistics, machine learning and enrichment	9	8	10	0	10	6	26	11
Single-cell and spatial analysis	0	0	0	0	14	10	16	7
Variant calling and annotation	1	0	0	0	9	4	3	5
Peak calling and epigenomics	0	0	0	0	4	2	5	6
Genomic intervals and sequence features	0	0	0	0	2	5	3	8
Read processing and alignment	0	0	0	0	4	4	1	2
Expression and differential testing	25	20	17	0	0	1	0	1
Phylogenetics	22	14	25	75	0	0	0	0
Sequence search, taxonomy and assembly	3	3	7	15	0	0	0	0
Tables and text	11	12	10	5	1	3	0	0
Script supplied as a dataset	15	20	10	5	51	52	42	41
Environment probe or set-up	5	3	8	0	3	8	4	18
Other methods	9	19	11	0	1	5	1	1
	GPT-5.5 (116)	Sol (59)	Luna (96)	DeepSeek (20)	GPT-5.5 (573)	Sol (729)	Luna (748)	DeepSeek (413)

Panel e of result_evaluation/figures/ed_fig4.pdf · image: result_evaluation/panels/ed4e.png

RATIONALE

- **Data.** Completed UDT jobs in traced Galaxy runs: 116, 59, 96 and 20 on BixBench-Verified-50 and 573, 729, 748 and 413 on CompBioBench for GPT-5.5, GPT-5.6 Sol, GPT-5.6 Luna and DeepSeek V4 Pro. Each job's UDT definition (container and command) is read from the archived trace.
- **Variables.** Each job gets one of 13 classes: the nine scientific method families of Fig. 4a, tables and text, other methods, environment probe or set-up, and script supplied as a dataset. Cells give the percentage of a model's completed UDT jobs on that benchmark.
- **Analysis.** Deterministic, ordered keyword rules: probes first, then the command when it contains the code, then a container named after a specific tool, then a script passed as an input dataset, then table and text handling. Descriptive only.
- **Reading the plot.** Each column sums to 100% before rounding; parentheses give jobs per column. "Script supplied as a dataset" marks jobs whose code sits in the Galaxy history, not in the request, so their method is not classified.

CONCLUSION

Many UDT jobs implemented recognisable scientific methods, which differed by benchmark. On BixBench-Verified-50, phylogenetics (14–25% for the GPT models; 75% of DeepSeek V4 Pro's 20 jobs) and expression and differential testing (17–25% for the GPT models) were the largest classes. On CompBioBench, 41–52% of each model's jobs ran a script supplied as a dataset, so their method is unknown from the request; classified jobs spanned statistics, single-cell, variant calling and other analyses. Probes and set-up took 0–18% of jobs. Classes come from keyword rules, not manual review, and each job gets one class.

Extended Data Fig. 5 | Token use by model, outcome and action count

This figure breaks the pooled token comparisons of Fig. 5a down by model configuration and outcome, and shows how input tokens grow with the number of actions. It asks whether the token gap holds for each model, and whether token use tracks score or correctness.

PANELS IN THIS FIGURE

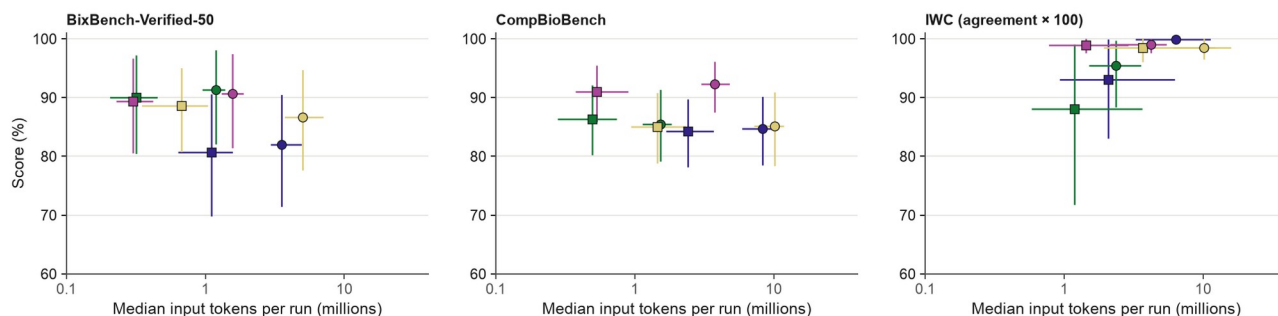
- **Extended Data Fig. 5a** Does higher input-token use go with higher scores across model configurations and conditions?
- **Extended Data Fig. 5b** Do incorrect runs use more input tokens than correct runs, model by model?
- **Extended Data Fig. 5c** How does a run's input-token use scale with its number of actions?

Extended Data Fig. 5a | Does higher input-token use go with higher scores across model configurations and conditions?

a Score and input tokens per model and condition

Points, mean score (IWC, output agreement) and median input tokens per run, with 95% intervals

● GPT-5.5 ● GPT-5.6 Sol ● GPT-5.6 Luna ● DeepSeek V4 Pro ■ Custom code ● Galaxy



Panel a of result_evaluation/figures/ed_fig5.pdf · image: result_evaluation/panels/ed5a.png

RATIONALE

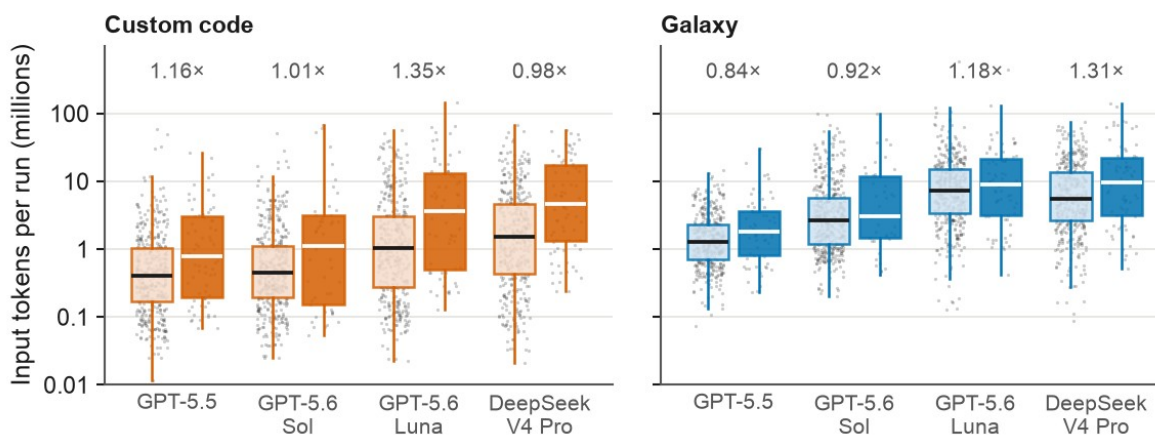
- **Data.** Scored primary runs with a token count, grouped by benchmark, model configuration and condition. The 24 groups hold 150 runs each on BixBench-Verified-50, 290–300 on CompBioBench and 30 on IWC (10 tasks, including host-read removal). BixBench-Verified-50 scores use the results-site grades, including the regrades of bix-53-q2 and bix-43-q2.
- **Variables.** x, median input tokens per run, including cached context (millions). y, mean score in percent; for IWC, mean output agreement $\times 100$. Colour marks the model; squares are custom code, circles Galaxy.
- **Analysis.** 95% cluster-bootstrap intervals (2,000 resamples; clusters are BixBench source capsules, otherwise tasks) for both the mean score and the median tokens. Descriptive only; no tests.
- **Reading the plot.** Log x-axis. Horizontal bars are token intervals and vertical bars score intervals. Compare a model's square and circle for the condition gap, and compare colours for model differences.

CONCLUSION

In all 12 model–benchmark pairs, Galaxy's median input exceeded custom code's; for example, 1.19M against 0.32M for GPT-5.5 on BixBench-Verified-50, and 3.75M against 0.53M for GPT-5.6 Sol on CompBioBench. On those two benchmarks, each model's scores in the two conditions differed by at most 2 points. Higher token use did not go with higher scores across models: GPT-5.6 Luna in Galaxy used the most input on both (5.0M and 10.1M) and scored 87% and 85%. Scores here include only runs with a token count.

Extended Data Fig. 5b | Do incorrect runs use more input tokens than correct runs, model by model?

b Input tokens of correct (light) and incorrect (solid) runs



Panel b of result_evaluation/figures/ed_fig5.pdf · image: result_evaluation/panels/ed5b.png

RATIONALE

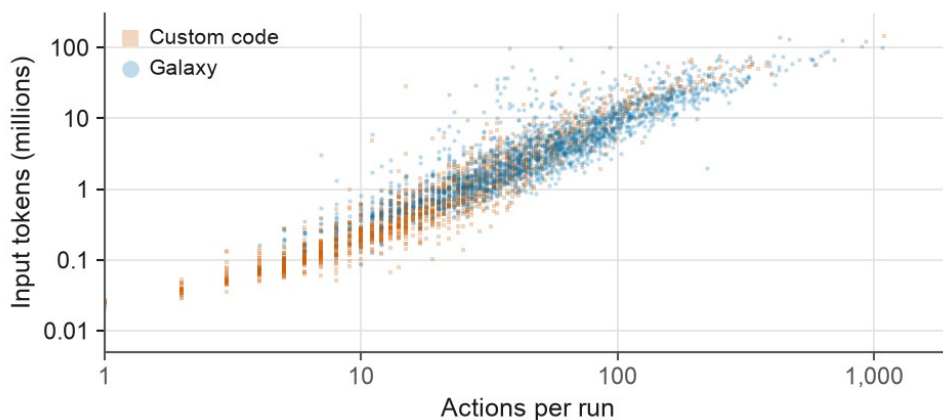
- **Data.** The 3,826 scored primary runs with a token count, split by condition, model configuration and outcome, and pooled over benchmarks. The ratios use replicate sets with both outcomes: 13–32 per model and condition. Outcomes use the results-site grades; the bix-43-q2 regrade adds two sets and changes which runs are correct in two others.
- **Variables.** Input tokens per run, including cached context (millions). Light boxes are correct runs and solid boxes incorrect runs; a run is correct when accepted or, for IWC, at ≥ 0.99 output agreement. The number above each pair is the incorrect / correct ratio.
- **Analysis.** Ratios are geometric means over sets of median incorrect / median correct input, the lower plot of Fig. 5a split by model. Intervals are 95% cluster-bootstrap intervals; sign-flip P values, Holm-adjusted over the eight tests, are in Source Data. Boxes are descriptive.
- **Reading the plot.** Log y-axis. Boxes show the middle 50% and the median, whiskers reach 1.5 times the interquartile range, and grey dots are runs. Orange is custom code; blue is Galaxy.

CONCLUSION

No model shows a clear difference. Within replicate sets, incorrect / correct ratios ranged from 0.98× to 1.35× in custom code and from 0.84× to 1.31× in Galaxy. Every 95% interval includes 1, though only just for GPT-5.6 Luna in custom code (1.35×, 0.998–1.85), and every Holm-adjusted P is at least 0.61. The pooled boxes place incorrect runs higher in all eight groups, but they mix tasks; the within-set ratios hold task and model fixed and are the fairer comparison.

Extended Data Fig. 5c | How does a run's input-token use scale with its number of actions?

C Input tokens against actions (all runs)



Panel c of result_evaluation/figures/ed_fig5.pdf · image: result_evaluation/panels/ed5c.png

RATIONALE

- **Data.** The 3,825 scored primary runs of panel b that have at least one recorded action, one point per run, with benchmarks and models pooled. Token counts come from the agent traces and action counts from the archive's per-run action table, as in Fig. 5b.
- **Variables.** x, actions per run: the agent's tool calls (shell commands, Galaxy interface calls, web searches or fetches, file operations). y, input tokens per run, including cached context (millions). Orange squares are custom code; blue circles are Galaxy.
- **Analysis.** Descriptive only; no fitted line, correlation or test. The panel shows the shape of the association, not a model of it.
- **Reading the plot.** Both axes are log scale, so equal distances are equal fold changes. Points are semi-transparent, so dense regions look darker.

CONCLUSION

Input tokens rise steeply with actions in both conditions, and the two conditions fall along one shared band. Custom-code runs dominate the low-action end, and Galaxy runs extend further towards high action counts. This matches Fig. 5b, where more actions account for much of Galaxy's extra input. Each later call carries the retained context, so cumulative input grows with actions by construction. The association is therefore not a decomposition of cost.

Extended Data Fig. 6 | Independent checks of the audits and of benchmark integrity

This figure tests two pieces of evidence behind the main results. First, it asks whether some runs reached benchmark answers during the run, and whether that changes the Galaxy–custom code comparison. Second, it asks whether an independent rater reproduces the failure-cause audit and the rule-based failure classes used in the main figures.

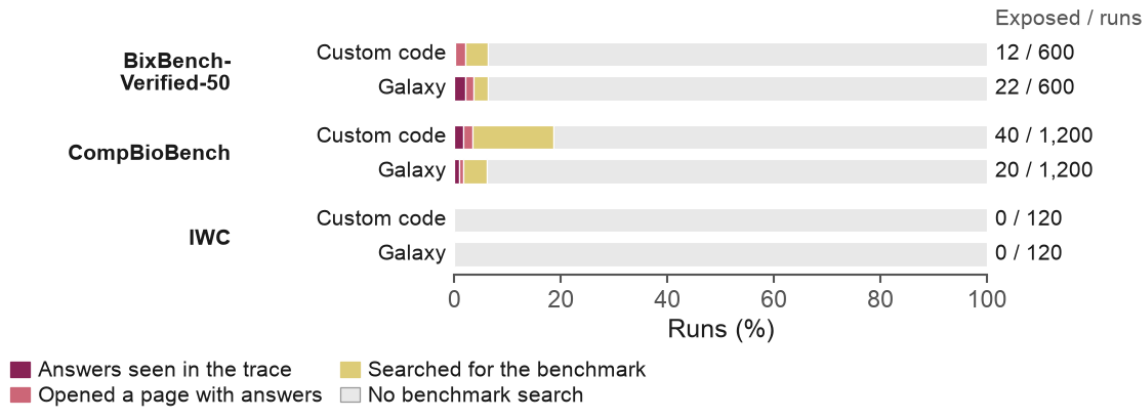
PANELS IN THIS FIGURE

- **Extended Data Fig. 6a** How many runs reached a source of benchmark answers during the run, and in which benchmark and condition?
- **Extended Data Fig. 6b** Does the Galaxy–custom code difference in runs correct change when runs that reached or searched for benchmark answers are removed?
- **Extended Data Fig. 6c** Does an independent rater assign the same primary failure cause as the original audit to incorrect BixBench-Verified-50 runs?
- **Extended Data Fig. 6d** Does an independent rater reproduce the rule-based failure classes of failed Galaxy requests?

Extended Data Fig. 6a | How many runs reached a source of benchmark answers during the run, and in which benchmark and condition?

a Runs that reached benchmark answers during the run

Highest tier per run, from the traces (web pages themselves are not logged)



Panel a of result_evaluation/figures/ed_fig6.pdf · image: result_evaluation/panels/ed6a.png

RATIONALE

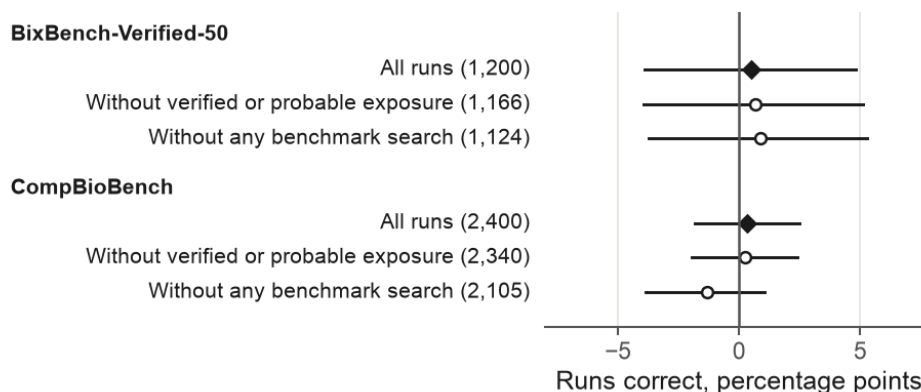
- **Data.** The panel covers all 3,840 scored primary runs: 600 per condition on BixBench-Verified-50, 1,200 on CompBioBench and 120 on IWC. A rule-based scan searched each trace for web searches, commands and connector calls that touched benchmark sources. Twelve CompBioBench Galaxy runs have no trace and are counted as no search.
- **Variables.** Each run takes its highest tier: *answers seen* (the reference answer, or other agents' recorded answers, appeared in a command or connector output); *opened a page with answers* (the public BixBench dataset or a published archive of agent traces, whose content is not logged); *searched for the benchmark*; or *no benchmark search*. A run is exposed if it falls in either of the first two tiers.
- **Analysis.** Descriptive only (counts and percentages).
- **Reading the plot.** Each bar is 100% of one benchmark × condition, split by tier. The numbers on the right are exposed runs over scored runs.

CONCLUSION

Of 3,840 scored runs, 94 (2.4%) were exposed. On BixBench-Verified-50, 12 of 600 custom-code runs and 22 of 600 Galaxy runs were exposed; on CompBioBench, 40 of 1,200 and 20 of 1,200; on IWC, none of 120 in either condition. Another 277 runs searched for the benchmark without opening an answer source. Searching was most common in CompBioBench custom-code runs (223 of 1,200 in any tier). DeepSeek V4 Pro accounts for 79 of the 94 exposed runs. Page content is not logged, so the middle tier is probable exposure, not confirmed exposure.

Extended Data Fig. 6b | Does the Galaxy–custom code difference in runs correct change when runs that reached or searched for benchmark answers are removed?

b Galaxy minus custom code without exposed runs



Panel b of result_evaluation/figures/ed_fig6.pdf · image: result_evaluation/panels/ed6b.png

RATIONALE

- **Data.** The panel uses scored runs on BixBench-Verified-50 (1,200) and CompBioBench (2,400). IWC is omitted because no IWC run was exposed. Each benchmark has three populations: all runs; runs outside the two exposure tiers (1,166 and 2,340); and runs with no benchmark search (1,124 and 2,105).
- **Variables.** The estimate is the share of Galaxy runs correct minus the share of custom-code runs correct, in percentage points, with runs pooled within each benchmark. A run is correct when its score is 1.
- **Analysis.** Intervals are 95% cluster-bootstrap percentile intervals. The bootstrap resamples analysis capsules for BixBench-Verified-50 and tasks for CompBioBench.
- **Reading the plot.** Filled diamonds are all runs and open circles are the reduced populations. Horizontal lines are 95% intervals. Points right of zero favour Galaxy.

CONCLUSION

Removing exposed runs left the condition difference almost unchanged. On BixBench-Verified-50 it was +0.5 points (95% interval -3.9 to 4.8) with all runs and +0.6 (-4.0 to 5.1) without exposed runs. On CompBioBench it was +0.3 (-1.8 to 2.5) and +0.2 (-2.0 to 2.4). Removing every run that searched gave +0.9 (-3.7 to 5.3) on BixBench-Verified-50 and -1.3 (-3.9 to 1.1) on CompBioBench. No difference was detected in any population. The removed runs are not a random subset (mostly DeepSeek V4 Pro runs), so these are sensitivity checks, not unbiased estimates.

Extended Data Fig. 6c | Does an independent rater assign the same primary failure cause as the original audit to incorrect BixBench-Verified-50 runs?

C Second rater for the failure-cause audit (BixBench-Verified-50)
45 incorrect runs; agreement 78%, Cohen's $\kappa = 0.56$; 96% counting secondary causes

Original audit	Validation	3	3	2	0	0
	Benchmark	3	27	0	0	0
	Knowledge	0	1	2	0	0
	Galaxy	1	0	0	1	0
	No answer	0	0	0	0	2
		Validation	Benchmark	Knowledge	Galaxy	No answer
		Second rater				

Panel c of result_evaluation/figures/ed_fig6.pdf · image: result_evaluation/panels/ed6c.png

RATIONALE

- **Data.** The sample is 45 BixBench-Verified-50 runs graded incorrect at the time of the audit (23 Galaxy, 22 custom code), stratified by the original audit's cause and by condition. Seven of them (six on bix-53-q2, one on bix-43-q2) are graded correct on the results site after the regrades of those two tasks, so the check measures agreement on the cause as audited. The second rater was an AI coder, blind to the audit, that read each run's transcript, submitted answer and reference answer.
- **Variables.** Rows give the original audit's primary cause and columns the second rater's. Causes are grouped as in Fig. 2d: Validation (no answer validation), Benchmark (specification or scoring), Knowledge, Galaxy (platform) and No answer. Cells count runs.
- **Analysis.** Percent agreement and Cohen's κ on the primary cause group. A second measure also counts a run as agreeing when one rater's secondary cause matches the other's primary cause.
- **Reading the plot.** Agreements lie on the diagonal (bold). Darker cells hold more runs.

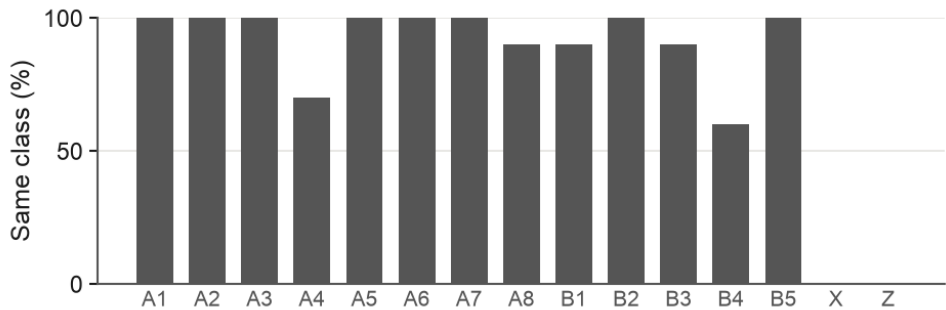
CONCLUSION

The second rater chose the same primary cause group in 35 of 45 runs (78%; $\kappa = 0.56$), or 96% when secondary causes count. It confirmed 27 of the 30 runs that the audit attributed to the benchmark, including all seven runs now graded correct after the regrades. Most disagreements were between Validation and Benchmark (3 runs each way). One of the two Galaxy-attributed runs was recoded as Validation. Agreement on the Knowledge, Galaxy and No answer groups rests on two or three runs each. This is an AI-assisted second rating, not human validation, and the original audit was also AI-assisted, so the two raters may share biases.

Extended Data Fig. 6d | Does an independent rater reproduce the rule-based failure classes of failed Galaxy requests?

d Second rater for the failure classes of failed requests

150 requests, 10 per class; agreement 92% for classified requests ($\kappa = 0.92$); the rule's improvement group was among the rater's in 89%



Rater judged the failure an agent error only in 3%; more than one improvement plausible in 35%

Panel d of result_evaluation/figures/ed_fig6.pdf · image: result_evaluation/panels/ed6d.png

RATIONALE

- **Data.** The sample is 150 failed Galaxy requests, 10 from each of 15 rule-based classes. A1–A8 are requests rejected before a job ran, B1–B5 are jobs that failed, X is other exceptions and Z is unclassified. An AI coder, blind to the rule's class, assigned a class and one or more changes that would plausibly have prevented the failure.
- **Variables.** Each bar is the percentage of a class's 10 requests that the rater put in the same class. The subtitle also compares the rater's prevention choices with the improvement group the rule links to each class, over the 120 requests whose class has a group (B3, X and Z have none).
- **Analysis.** Percent agreement and Cohen's κ over the 130 requests in classes A1–B5. The panel also gives the share of requests judged agent error only and the share with more than one plausible improvement.
- **Reading the plot.** A full bar means all 10 requests were reproduced. X and Z have no bars because they have no rule definition to reproduce.

CONCLUSION

The rater reproduced the rule's class for 92% of classified requests ($\kappa = 0.92$). It matched every request in 8 of 13 classes. Agreement was lowest for B4, input format (60%), and A4, parameter value or datatype (70%). For 89% of the requests whose class has an improvement group (107 of 120), the rule's group was among the rater's choices. This match is lenient, because the rater could name several improvements and did so in 35% of requests. It judged only 3% of failures to be agent error alone. This is an AI-assisted second rating, not human validation.

Extended Data Fig. 7 | Verification, recovery and selected cases

This figure looks at what agents did inside their runs. It asks whether they checked their own analyses, whether they re-ran failed steps without error, and what such checks look like in two worked cases. These results bear on the paper's claim that verification, more than execution, limits reliable agent analyses.

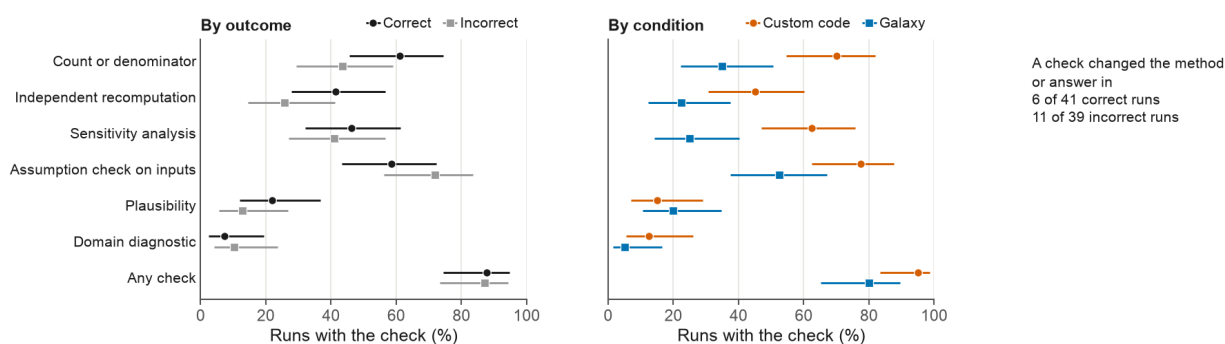
PANELS IN THIS FIGURE

- **Extended Data Fig. 7a** How often did agents check their own analysis, and did checks differ between correct and incorrect runs or between conditions?
- **Extended Data Fig. 7b** How often was a failed step re-run without error later in the same run, and did those runs end correct more often?
- **Extended Data Fig. 7c** What happens when the Galaxy interface detects that a request would not set the parameter the agent asked for?
- **Extended Data Fig. 7d** In a task where read-end artefacts mimic an alternate allele, which runs ran a read-position diagnostic, and which were accepted?

Extended Data Fig. 7a | How often did agents check their own analysis, and did checks differ between correct and incorrect runs or between conditions?

a Verification checks in 80 coded runs (coder blind to the grade)

10 runs per benchmark × condition × outcome; 95% Wilson intervals



Panel a of result_evaluation/figures/ed_fig7.pdf · image: result_evaluation/panels/ed7a.png

RATIONALE

- **Data.** The sample is 80 primary runs, drawn at 10 per benchmark (BixBench-Verified-50, CompBioBench) × condition × outcome, with at most one run per task in each stratum; IWC is not sampled. The outcome plotted is the current grade: one run drawn as incorrect (bix-53-q2, GPT-5.6 Luna, Galaxy, replicate 1) is correct after that task's regrade. AI coders read condensed transcripts, blind to the grade but not to the condition.
- **Variables.** Six kinds of check, plus *any check*, each counted only when the agent explicitly tested something its answer depended on. The left plot compares correct and incorrect runs (41 and 39) and the centre plot custom code and Galaxy (40 each); the text on the right gives the runs in which a check changed the method or the answer.
- **Analysis.** Percentages with 95% Wilson intervals. Descriptive only; no tests. One plausibility code was removed because it recorded a download of the BixBench answer file.
- **Reading the plot.** Each point is the share of runs with the check, and each line is its 95% interval.

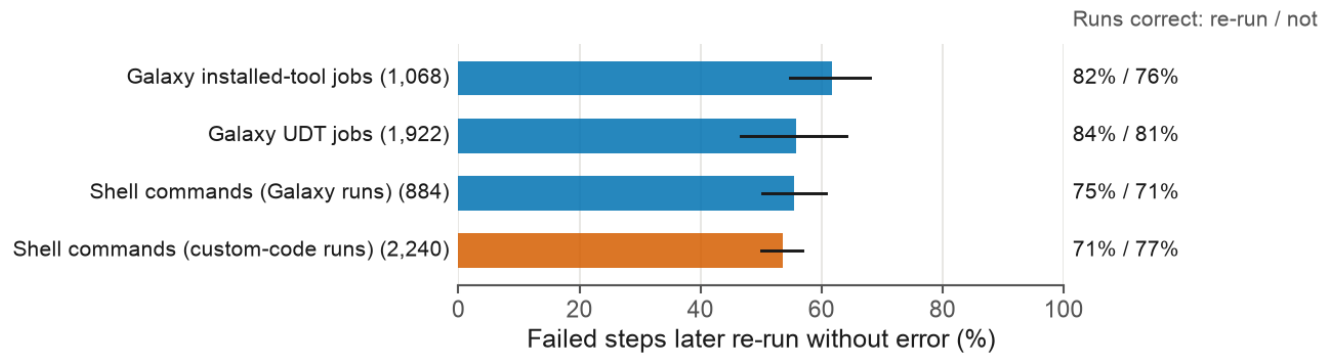
CONCLUSION

Checks were about as common in correct as in incorrect runs (any check, 87.8%, 95% interval 74.5–94.7, vs 87.2%, 73.3–94.4). Count checks (61.0% vs 43.6%) and independent recomputation (41.5% vs 25.6%) were more frequent in correct runs, but the intervals overlap. Custom-code runs had more checks than Galaxy runs (any check, 95.0%, 83.5–98.6, vs 80.0%, 65.2–89.5). The intervals do not overlap for count checks (70.0% vs 35.0%) or sensitivity analyses (62.5% vs 25.0%). A check changed the method or answer in 6 of 41 correct and 11 of 39 incorrect runs. The sample was drawn balanced, not representative of all runs, so these are not population rates or causal effects.

Extended Data Fig. 7b | How often was a failed step re-run without error later in the same run, and did those runs end correct more often?

b Failed steps later re-run without error in the same run

Galaxy jobs by tool; shell commands by the analysis program or script they ran (inline code excluded)



Panel b of result_evaluation/figures/ed_fig7.pdf · image: result_evaluation/panels/ed7b.png

RATIONALE

- **Data.** The panel covers 6,114 failed steps in scored primary runs on all three benchmarks, 53 of them on the IWC host-read removal task. These are 1,068 installed-tool and 1,922 UDT Galaxy jobs in the error state, and 884 failed shell commands in Galaxy runs and 2,240 in custom-code runs. A shell command counts only if it ran a named analysis program or script.
- **Variables.** A Galaxy job is resolved if a later job of the same tool completed (for UDTs, any later UDT job), and a shell command if a later command of the same program or script exited 0. The right-hand column is the share of failed steps whose run ended correct, for resolved and unresolved steps; it is weighted by failed step, so runs with many failures count more.
- **Analysis.** Intervals are 95% cluster-bootstrap intervals; clusters are analysis capsules for BixBench-Verified-50 and tasks otherwise. The right-hand shares are descriptive.
- **Reading the plot.** Blue bars are Galaxy runs and the orange bar is custom-code runs. Black lines are 95% intervals.

CONCLUSION

In every channel, 54–62% of failed steps were later re-run without error. The shares were 62% (95% interval 54.7–68.0) for installed-tool jobs and 56% (46.7–64.1) for UDT jobs. For shell commands they were 56% (50.2–60.8) in Galaxy runs and 54% (50.1–56.9) in custom-code runs. The intervals overlap and no formal comparison was made. Final correctness differed by less than 7 points between resolved and unresolved failures, and not in a consistent direction (82% vs 76% for installed-tool jobs, 71% vs 77% for custom-code shell commands). Resolution means the step later ran without error, not that the error was understood or the result was right.

Extended Data Fig. 7c | What happens when the Galaxy interface detects that a request would not set the parameter the agent asked for?

C A parameter check that stopped a job

Task	bix-43-q4, GPT-5.5, Galaxy, replicate 1 (Reactome enrichment)
Tool	gseapy_enrichr 0.1.0+galaxy2
First request	nested: gene_sets = {source, library_name, organism}
Interface check	gene_sets library_name: requested Reactome_2022, Galaxy would bind (empty); the job was not submitted
Agent response	Resubmitted with flat parameter keys
Second request	flat: gene_sets library_name = Reactome_2022; parameters matched; job ran

Panel c of result_evaluation/figures/ed_fig7.pdf · image: result_evaluation/panels/ed7c.png

RATIONALE

- **Data.** One selected run: bix-43-q4 (Reactome enrichment), GPT-5.5, Galaxy condition, replicate 1. The script hard-codes this run and, from its trace, extracts the first gseapy_enrichr request that the interface stopped with a parameter mismatch, and the next request to that tool that completed (trace lines 63 and 70).
- **Variables.** The rows give the task, the tool and version, and the structure of the first request. They then give the interface check (requested value against the value Galaxy would bind), the agent's response and the second request.
- **Analysis.** None; this is a worked example. The repository records no rule for choosing this run; it illustrates the mechanism.
- **Reading the plot.** Read the rows from top to bottom as the order of events in the run.

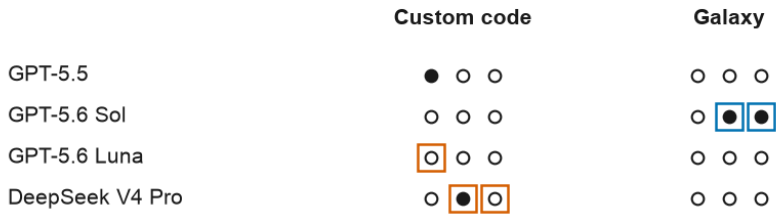
CONCLUSION

In this run, the agent first put the gene-set library inside a nested structure. The interface found that Galaxy would bind gene_sets|library_name to an empty value instead of Reactome_2022, so it did not submit the job. The agent resubmitted with flat parameter keys, the parameters matched and the job ran. The case shows that the interface check can stop a job that would otherwise run silently with a wrong parameter. It is one selected case. It does not estimate how often this happens, and matching parameters do not show that the analysis was scientifically appropriate.

Extended Data Fig. 7d | In a task where read-end artefacts mimic an alternate allele, which runs ran a read-position diagnostic, and which were accepted?

d A selected case: variant-status-q1 (CompBioBench)

Read-end artefacts mimic an alternate allele; answers are not shown



● accepted ○ rejected ○ ran a read-position diagnostic (5 runs)
4 of 24 runs accepted. GPT-5.6 Sol in Galaxy ran the diagnostic as a UDT in replicates 2 and 3, and both were accepted; 3 custom-code runs also ran it (1 accepted).

Panel d of result_evaluation/figures/ed_fig7.pdf · image: result_evaluation/panels/ed7d.png

RATIONALE

- **Data.** All 24 primary runs on one selected CompBioBench task, variant-status-q1 (4 model configurations × 2 conditions × 3 replicates). The task is one of the worked cases from the failure-cause audit, in which read-end artefacts were taken for alleles and contrasted with a read-position audit.
- **Variables.** Filled circles are accepted runs and open circles rejected runs. A box marks a run that ran a read-position diagnostic (blue, a UDT in Galaxy; orange, a command in custom code). Runs are flagged by a text pattern (read position, cycle or end) in a UDT call or shell command, so diagnostics inside separately written scripts may be missed.
- **Analysis.** Descriptive only (counts).
- **Reading the plot.** Rows are model configurations. Within each condition, the three markers are replicates 1–3. Answers are not shown.

CONCLUSION

Four of 24 runs were accepted. Five runs ran the read-position diagnostic, and three of these were accepted. GPT-5.6 Sol in Galaxy ran it as a UDT in replicates 2 and 3, and both were accepted. Of the three custom-code runs with the diagnostic, one was accepted (DeepSeek V4 Pro, replicate 2). Only one of the 19 runs without a detected diagnostic was accepted. The case shows that a UDT let an agent run a domain diagnostic inside Galaxy. With five diagnostic runs on one selected task, it cannot show that the diagnostic causes acceptance or how often agents run such checks.